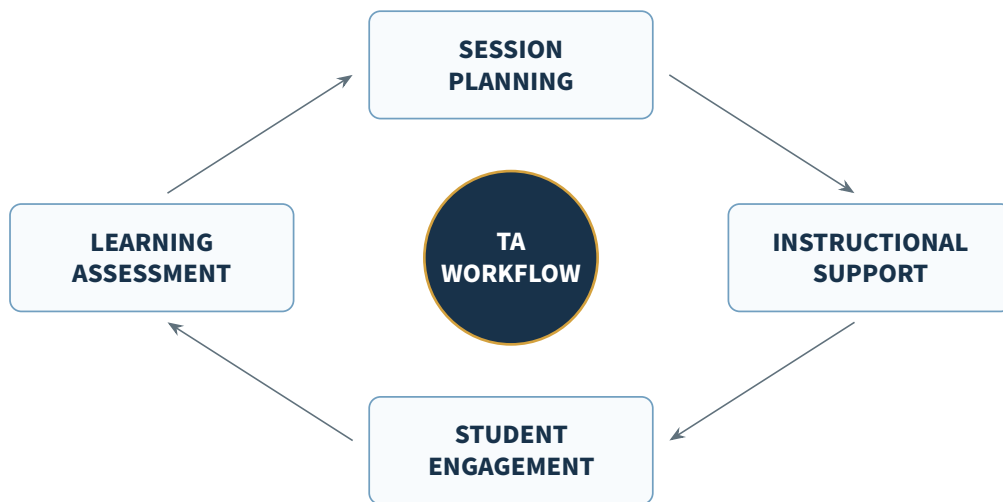


Algorithms & Data Structures

Teaching Assistant Guide



COURSE OUTLINE

C++ examples, guided laboratories, and comparative projects connect data-structure choices with correctness and performance.

PREPARED BY

Reeyan Afzal Khan

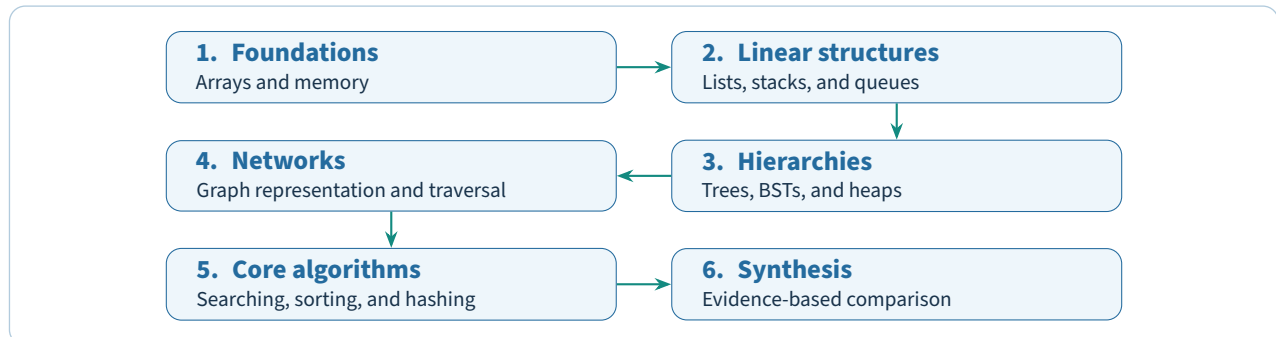
Graduate Research Assistant

Learning Path

Course Map	ii
Stage 1 Foundations	1
Chapter 1 Introduction to Data Structures	2
Lab 1 of 11 — Programming foundations and structured records	8
Chapter 2 Arrays and Two-Dimensional Arrays	10
Lab 2 of 11 — Arrays, multidimensional storage, and matrices	15
Stage 2 Linear Dynamic Structures	17
Chapter 3 Linked Lists I: Singly and Doubly Linked Lists	18
Lab 3 of 11 — Singly and doubly linked lists	23
Chapter 4 Linked Lists II: Circular Linked Lists	25
Lab 4 of 11 — Circular linked-list operations	30
Chapter 5 Stacks	31
Lab 5 of 11 — Stack ADT and expression processing	36
Chapter 6 Queues	38
Lab 6 of 11 — Linear, linked, circular, and priority queues	43
Stage 3 Trees and Hierarchies	44
Chapter 7 Trees I: Terminology, Traversal, and Binary Trees	45
Lab 7 of 11 — Tree representation and traversal	50
Chapter 8 Trees II: BSTs, Heaps, and Height Balancing	51
Lab 8 of 11 — BSTs, heaps, and balancing structures	56
Stage 4 Graphs and Traversal	57
Chapter 9 Graphs	58
Lab 9 of 11 — Graph representation, traversal, and spanning trees	63
Stage 5 Core Algorithms and Fast Access	65
Chapter 10 Searching and Sorting	66
Lab 10 of 11 — Searching and sorting experiments	74
Chapter 11 Hashing	76
Lab 11 of 11 — Hashing and collision resolution	81
Stage 6 Course Project and Synthesis	82
Chapter 12 Final Project: Algorithm Comparison	83
Chapter A Quick Reference	89

Course Map

The course progresses from data organization to algorithm selection. Five stages build core skills; the final project compares algorithms with evidence.

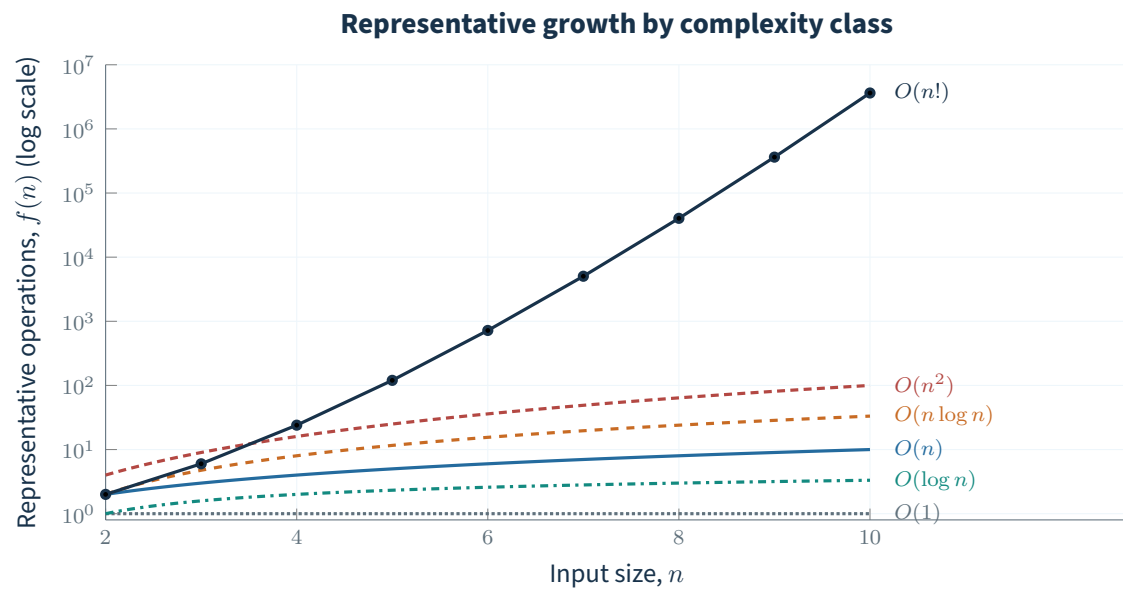


Stage	Main structures	Central question
1	Primitive/non-primitive structures, arrays	How is data represented and accessed in memory?
2	Linked lists, circular lists, stacks, queues	How can pointer-based structures support dynamic or constrained access?
3	Trees, binary trees, BSTs, heaps, balancing	How can hierarchy improve organization and searching?
4	Graphs, adjacency structures, BFS/DFS, spanning trees	How can general relationships between objects be represented and traversed?
5	Searching, sorting, hashing	How do algorithm choice and data organization affect running time?
6	Final comparison project	How can two related algorithms be compared fairly using complexity, operation counts, runtime, and practical use cases?

Complexity vocabulary used across the course

Big-O notation describes an asymptotic upper bound on how resource use grows with input size n ; it is not an exact operation count. The table and chart use representative functions $f(n)$ to compare growth rates consistently.

Class	Growth rate	Function plotted	Typical examples
$O(1)$	Constant	$f(n) = 1$	Array access; hash computation.
$O(\log n)$	Logarithmic	$f(n) = \log_2 n$	Binary search; balanced-BST lookup.
$O(n)$	Linear	$f(n) = n$	Linear search; linked-list traversal.
$O(n \log n)$	Linearithmic	$f(n) = n \log_2 n$	Merge sort; heap sort.
$O(n^2)$	Quadratic	$f(n) = n^2$	Bubble sort; selection sort.
$O(n!)$	Factorial	$f(n) = n!$	Permutation enumeration; brute-force traveling salesperson search.



The logarithmic vertical scale keeps all six functions visible. Each curve follows the representative function listed in the table; actual running time depends on the implementation and hardware.

Key idea

Factorial complexity, $O(n!)$, arises when an algorithm examines every ordering of n items. It becomes impractical very quickly: $10! = 3,628,800$ possible orderings, while $15!$ exceeds one trillion. Brute-force permutation problems and exhaustive solutions to the traveling salesperson problem are common examples.

Stage 1

Foundations

Introduction to Data Structures

Chapter at a glance

Before you start: Basic C++ variables and control flow are useful, but the chapter starts from the course definition of data and data structures.

By the end, you can: classify primitive and non-primitive structures, distinguish linear and non-linear organization, name the common operations, explain correctness/time/space concerns, and use a C++ `struct` to represent a record.

A data structure is not just a container. It is a decision about how data will be organized, related, stored, and accessed in memory. That decision affects both the structural and functional design of a program.

Key idea

The course summarizes the relationship as

$$\text{Program} = \text{Algorithm} + \text{Data Structure}.$$

An algorithm describes the steps. A data structure determines how the data used by those steps is arranged.

1.1 From data to a data structure

The Week 1 slides separate two ideas:

Data A collection of raw facts and figures.

Data structure A way to organize, store, and relate data in memory.

Intuition

Five student ages are data. Storing them as an array is a structural choice. Storing each student's ID, name, age, and course together in one record is a different structural choice. The values may be similar, but the relationships you need to preserve are different.

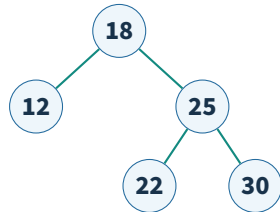
Figure 1.1 makes the point concrete. The five values never change, but each layout answers a different question cheaply and a different question expensively. Choosing the structure *is* choosing which operations will be fast.

ARRAY

reach item 4 $O(1)$
 insert in middle $O(n)$

LINKED LIST

reach item 4 $O(n)$
 insert once located $O(1)$

SEARCH TREE

search by value $O(\log n)$
 ordered scan $O(n)$, sorted

Figure 1.1. The same five values under three organisations. Nothing about the data changed — only the relationships that were stored alongside it — yet the cost of a lookup, an insertion, and an ordered scan changes completely.

1.2 Classification of data structures

The slides divide data structures into two broad categories: **primitive** and **non-primitive**.

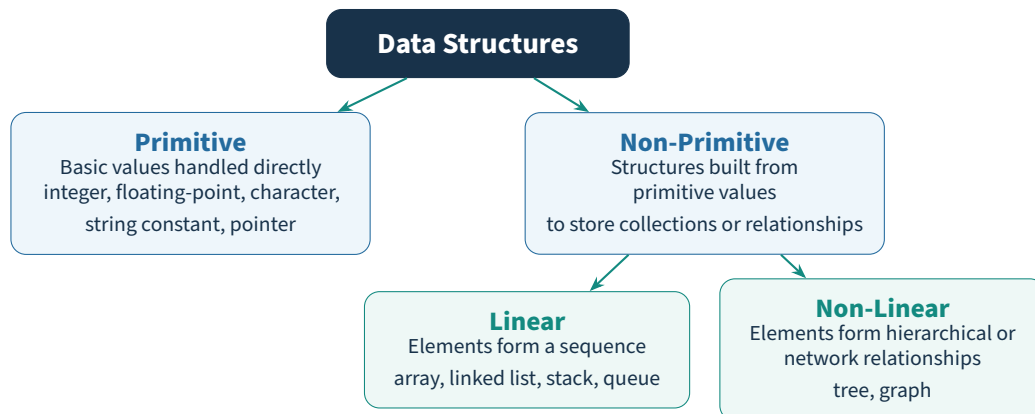


Figure 1.2. The course classification of data structures. Primitive types hold single values; non-primitive structures organise collections, and split further by whether the elements form a single sequence or a branching relationship.

1.2.1 Primitive data structures

The course describes primitive data structures as basic structures directly manipulated by machine-level instructions. The slides list integers, floating-point numbers, character constants, string constants, and pointers as examples.

Typical operations highlighted for primitive structures are:

- create;

- select or access;
- update;
- destroy or delete.

Clarification beyond the slide wording

The Week I classification is the course framing. In modern C++, types such as `std::string` are library classes rather than built-in primitive machine types. Keep the slide classification for this course, but be aware that language-level terminology can be more precise.

1.2.2 Non-primitive data structures

Non-primitive structures are described as structures derived from primitive data structures. They can organize homogeneous items of the same type or heterogeneous items of different types, and they can represent relationships among items.

1.2.3 Linear data structures

In a linear structure, elements form a sequence or linear series. The slides give stacks, queues, and linked lists as common examples. Arrays, introduced in the next chapter, are another central linear structure in the course.



one predecessor, one successor: a single chain

Figure 1.3. In a linear structure every element has at most one predecessor and one successor, so there is exactly one path through the data.

1.2.4 Non-linear data structures

A non-linear structure allows one item to connect to several other items. The slides emphasize hierarchical or parent-child relationships and give **trees** and **graphs** as the main examples.

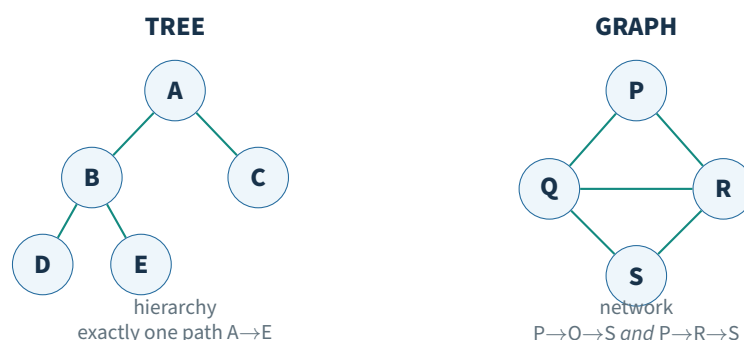


Figure 1.4. Two non-linear shapes. A tree imposes a strict hierarchy with one path between any two nodes; a graph allows cycles and several competing paths, which is why graph traversal must remember what it has already visited.

1.3 Common operations

For non-primitive structures, the Week I slides list the following common operations:

Operation	Meaning in the course context
Traversal	Visit elements systematically.
Insertion	Add a new element.
Selection	Access or choose a particular element.
Searching	Find an element that matches a condition or key.
Sorting	Rearrange elements into a specified order.
Merging	Combine structures or collections.
Destroy/Delete	Remove elements or release the structure.

These operation names recur throughout the course. The important change from chapter to chapter is not the operation itself, but how the structure makes the operation easier or harder.

1.4 Primitive vs non-primitive structures

Feature	Primitive	Non-primitive
Meaning	Basic built-in or basic-value type in the course classification	Built using primitive types
Examples	int, float, char, pointer	array, linked list, tree, graph
Structure	Simple	Can store groups or relationships
Memory use	Usually fixed/simple	Can be static or dynamic
Use case	Single value	Grouped or connected data

1.5 Three characteristics of a good data-structure implementation

The Week I slides highlight three characteristics.

- Correctness** The implementation must implement its interface correctly.
- Time complexity** The running or execution time of operations should be as small as possible for the problem.
- Space complexity** The memory used by the operations should be as small as possible for the problem.

Key idea

A data structure is useful because of the operations it supports. Saying “a linked list is dynamic” or “an array gives direct access” matters because those properties change the cost and behavior of insertion, deletion, searching, and traversal.

1.6 C++ practical: structures (struct)

The slides introduce struct as a user-defined C/C++ data type that groups items of different types into a single record.

```
struct Student {  
    int id;  
    string name;  
    int age;  
    string course;  
};
```

Each variable of type Student contains all four fields.

Worked example

Suppose one student has ID 1001, name “Amina”, age 20, and course “Computer Science”. These values have different primitive or library types, but they belong to one logical record. A struct preserves that relationship.

```
Student s;  
s.id = 1001;  
s.name = "Amina";  
s.age = 20;  
s.course = "Computer Science";  
  
cout << s.id << " " << s.name << " "  
    << s.age << " " << s.course << '\n';
```

1.7 Putting records into an array

The Week 1 lab asks for at least five student records, then display, search, and update operations. One direct representation is an array of Student records.

```
const int N = 5;  
Student students[N] = {  
    {1001, "Amina", 20, "CS"},  
    {1002, "Ben", 21, "IT"},  
    {1003, "Chen", 20, "CS"},  
    {1004, "Devi", 22, "IS"},  
    {1005, "Ehsan", 21, "IT"}  
};
```

Searching by ID can be performed by examining records one by one.

```
int target = 1003;  
for (int i = 0; i < N; ++i) {  
    if (students[i].id == target) {  
        cout << "Found: " << students[i].name << '\n';  
        break;  
    }  
}
```

Check your understanding

Classify each as primitive or non-primitive using the course framing: an integer, a pointer, an array of integers, a linked list, a graph. Then name one operation whose efficiency could depend strongly on the chosen structure.

Chapter recap

- A data structure organizes, stores, and relates data in memory.
- The course divides structures into primitive and non-primitive categories.
- Non-primitive structures can be linear or non-linear.
- Core operations include traversal, insertion, selection, searching, sorting, merging, and deletion.
- Correctness, time complexity, and space complexity are central implementation concerns.
- C++ `struct` groups different fields into one record and can be combined with arrays for record-based applications.

PRACTICAL MILESTONE

Lab 1 of 11

Programming foundations and structured records

Source alignment — Official Lab Manual — Lab I

Objective. Revise programming prerequisites needed later in the course. The manual explicitly names structures, unions, pointers, and 1D arrays.

Observed outcomes. Understand data structures and their types; use data structures for unseen problems; select an appropriate structure for a real-world problem.

Problem directions. Basic C/C++ programs using loops and structures; programs on unions and 1D arrays; a small database-style application using structure, pointer, and array.

Build a small record-based application

Coverage. Chapter 1.

Source files. Lab01/Task01.cpp — the student-record application.

Goal. Store, display, search, and update at least five student records using a structure, an array, and a pointer.

The lecture task asks for at least five student records containing ID, name, age, and course, followed by display, search-by-ID, and update operations. This also satisfies the lab manual's database-style direction.

```
#include <iostream>
#include <string>
using namespace std;

struct Student {
    int id;
    string name;
    int age;
    string course;
};

void display(const Student students[], int n) {
    for (int i = 0; i < n; ++i) {
        cout << students[i].id << " | "
              << students[i].name << " | "
              << students[i].age << " | "
              << students[i].course << '\n';
    }
}

int findById(const Student students[], int n, int id) {
    for (int i = 0; i < n; ++i)
        if (students[i].id == id) return i;
    return -1;
}
```

Try it yourself

Add `updateStudent()` and use a pointer to the selected record once the matching index is known. Then add one small 1D-array statistic such as the average student age.

How to use this lab

The manual names *union* as prerequisite review but does not provide detailed union theory in the supplied text. Treat union exercises as revision work directed by your instructor rather than as a new data-structures topic.

Verification checklist

- At least five records are stored and displayed correctly.
- A valid ID is found; an invalid ID returns a clear “not found” result.
- Updating one record changes only the intended record.
- Array bounds are respected.
- The role of a pointer versus an array index can be explained.

Lab deliverable

Submit the C++ program, sample output for successful/unsuccessful search, and a short note identifying the data structure used for the collection and the structure used for each record.

Arrays and Two-Dimensional Arrays

Chapter at a glance

Before you start: Know the basic distinction between primitive and non-primitive data structures from Chapter 1.

By the end, you can: define an array, explain contiguous storage and index-based access, distinguish 1D and 2D arrays, trace row/column access, and solve matrix-style tasks with nested loops.

An **array** stores a collection of similar data items of the same type. The Week II slides emphasize three properties: same-type elements, contiguous memory locations, and random access through index positions.

Remember the sequence **Store** → **Access** → **Process**. Arrays make the access step simple because an index identifies the position directly.

2.1 What is an array?

Consider the one-dimensional array

[12, 25, 18, 30, 22].

Its indexes are 0, 1, 2, 3, 4 in normal C++ indexing.

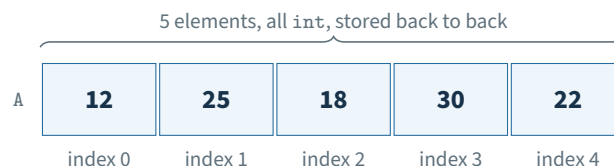


Figure 2.1. A one-dimensional array. The index is a position, not a value: index 3 and the value 30 are different things stored in different places.

2.1.1 Array properties from the slides

- Every element has the same data type and therefore the same element size.
- Elements are stored at contiguous memory locations.
- The first element occupies the smallest memory location in the block.
- An element can be accessed directly from its index because its address can be calculated from the base address and element size.

2.2 Address calculation

The slides state that the address of an array element can be calculated from the base address and element size. A standard expression for a one-dimensional, zero-based array is

$$\text{address}(A[i]) = \text{base}(A) + i \times \text{sizeof}(A[0]).$$

Worked example

If the base address is 1000 and each integer occupies 4 bytes, then the address of $A[3]$ is

$$1000 + 3(4) = 1012.$$

This arithmetic is the reason direct index access does not require traversing the earlier elements.

Figure 2.2 shows why that single multiplication is the whole story. The machine never looks at $A[0]$, $A[1]$, or $A[2]$ on the way to $A[3]$; it computes one address and reads it.

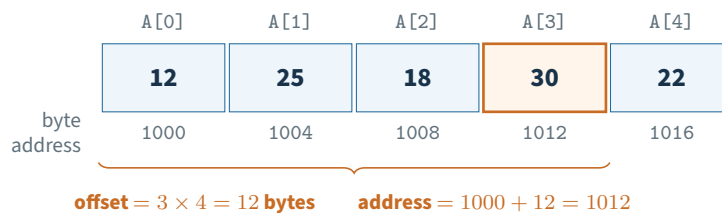


Figure 2.2. Contiguous storage turns indexing into arithmetic. Because every element is the same size, the address of any element is one multiply-and-add away from the base address — so reaching $A[3]$ costs the same as reaching $A[0]$.

Clarification beyond the slide wording

The formula above is an explanatory expansion of the slide statement about base address and element size. The slides themselves do not provide a numeric address formula.

2.3 One-dimensional vs two-dimensional arrays

A 1D array forms a single indexed sequence. A 2D array stores values in rows and columns and is commonly used for tables, grids, and matrices.

	col 0	col 1	col 2	
row 0	1	2	3	
row 1	4	5	6	← $\text{matrix}[1][2] = 6$
row 2	7	8	9	

Figure 2.3. A 3×3 array. The first index selects the row, the second selects the column, so $\text{matrix}[1][2]$ is row 1, column 2 — the value 6.

Clarification beyond the slide wording

A 2D array is a convenient *notation*; memory itself is still one flat sequence of bytes. C++ stores the rows one after another, an arrangement called **row-major order** (Figure 2.4). This is why $\text{matrix}[r][c]$ can still be resolved with arithmetic, and why walking a matrix row by row is friendlier to the cache than walking it column by column.

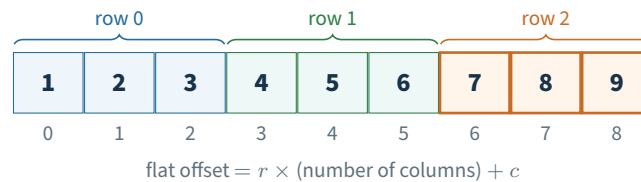


Figure 2.4. Row-major order: the rows of a two-dimensional array are laid end to end in linear memory. The element `matrix[r][c]` sits at flat offset $r \times 3 + c$, which is how two indices collapse back into one address.

In C++:

```
int matrix[3][3] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

cout << matrix[1][2]; // 6
```

The first index chooses the row; the second chooses the column.

2.4 Processing a 2D array with nested loops

The Week II summary explicitly connects 2D arrays with nested loops.

```
for (int r = 0; r < 3; ++r) {
    for (int c = 0; c < 3; ++c) {
        cout << matrix[r][c] << ' ';
    }
    cout << '\n';
}
```

The outer loop changes rows, while the inner loop visits the columns within the current row.

Worked example

To compute the sum of every element in a 3×3 matrix, initialize `sum = 0`, visit each pair `[r][c]`, and add the value. The traversal touches each matrix cell once.

2.5 Why arrays are useful

The Week II slides give three practical motivations:

- multiple values can be stored under one variable name;
- multiple values can be processed quickly and systematically;
- searching and sorting operations can be applied naturally to the indexed sequence.

Arrays are especially useful when the number of elements is known and direct index access is important.

2.6 The cost of an insertion

Contiguity is what makes access cheap, and it is also what makes insertion expensive. To place a value in the middle of a full-to-the-left array, every element from that position onwards must move up one slot to open a gap (Figure 2.5). Deletion is the mirror image: the survivors must close the gap.

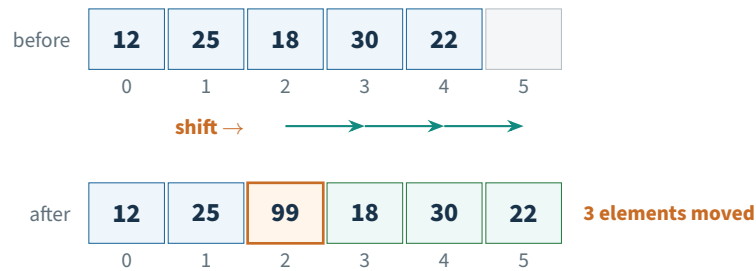


Figure 2.5. Inserting 99 at index 2. Because the elements must stay contiguous, the tail of the array shifts right one position before the new value can be written. Insertion near the front of a large array therefore costs $O(n)$ moves, even though *finding* index 2 costs nothing.

2.7 Array access vs later pointer-based access

The next chapter introduces linked lists. The contrast begins here.

Question	Array	Linked-list preview
How do I reach item 4?	Use an index directly.	Start at the head and follow links.
Memory layout	Contiguous.	Non-contiguous is allowed.
Size	Typically fixed for a raw C++ array.	Nodes can be allocated dynamically.
Extra pointer memory	Not needed per element.	Required for links.

2.8 Matrix multiplication as a lab-style problem

The Week II material explicitly points to multi-dimensional array problems such as matrix multiplication. If A is $m \times k$ and B is $k \times n$, then the product C is $m \times n$ with

$$C[i][j] = \sum_{t=0}^{k-1} A[i][t]B[t][j].$$

```
for (int i = 0; i < m; ++i) {
    for (int j = 0; j < n; ++j) {
        C[i][j] = 0;
        for (int t = 0; t < k; ++t) {
            C[i][j] += A[i][t] * B[t][j];
        }
    }
}
```

Clarification beyond the slide wording

The matrix-multiplication formula and loop structure are an implementation expansion of the Week II lab prompt, which asks for multidimensional-array problems such as matrix multiplication but does not spell out the algorithm.

Common mistake

Do not confuse an index with the stored value. In the array `[12, 25, 18, 30, 22]`, index 3 is the position and 30 is the value at that position.

Check your understanding

For a 4×5 integer matrix, what do the two indexes in `A[2][4]` represent? Why can a raw array element be accessed directly without walking through earlier elements?

Chapter recap

- Arrays store same-type elements in contiguous memory.
- Indexing gives direct access to an element.
- Address calculation depends on base address, index, and element size.
- 2D arrays represent rows and columns.
- Nested loops are the standard course pattern for matrix-style processing.
- Multidimensional arrays support practical tasks such as matrix multiplication.

PRACTICAL MILESTONE

Lab 2 of 11

Arrays, multidimensional storage, and matrices

Source alignment — Official Lab Manual — Lab II

Objective. Understand the array as a basic prerequisite for more complex data structures.

Outcomes. Explain how arrays are stored in memory and use arrays to solve programming problems.

Problem directions. Real-world scenarios involving arrays; multidimensional arrays; matrix-related problems such as matrix multiplication.

Process and multiply matrices

Coverage. Chapter 2.

Source files. Lab02/Task04.cpp (matrix multiplication, transpose, row/column sums).

Task01–Task03 cover matrix rotation, median, and maximal area.

Goal. Multiply two matrices and explain what each of the three nested loops controls.

```
#include <iostream>
using namespace std;

int main() {
    const int R = 2, K = 3, C = 2;
    int A[R][K] = {{1,2,3}, {4,5,6}};
    int B[K][C] = {{7,8}, {9,10}, {11,12}};
    int P[R][C] = {};

    for (int i = 0; i < R; ++i)
        for (int j = 0; j < C; ++j)
            for (int t = 0; t < K; ++t)
                P[i][j] += A[i][t] * B[t][j];
}
```

Try it yourself

Add two of the following: row sums, column sums, transpose, largest element, diagonal sum, or a check for whether a square matrix is symmetric.

Worked example

For matrix multiplication, the result size is $R \times C$. Each output cell $P[i][j]$ accumulates products across the shared dimension K .

Verification checklist

- Validate that the number of columns in A equals the number of rows in B .
- Print both inputs and the output matrix with clear row/column formatting.
- Trace one output cell by hand and match it to the program result.
- Explain what each nested loop controls.

Lab deliverable

Submit source code, output, and one hand-worked multiplication cell showing the same arithmetic performed by the innermost loop.

Stage 2

Linear Dynamic Structures

Linked Lists I: Singly and Doubly Linked Lists

Chapter at a glance

Before you start: Know how arrays store data contiguously and provide direct index access.

By the end, you can: explain nodes, head pointers, and address fields; compare arrays and linked lists; distinguish singly and doubly linked lists; and trace insertion, deletion, and traversal.

A linked list is a linear data structure made of a sequence of **nodes**. Each node stores data and one or more addresses that connect it to adjacent nodes. Unlike an array, the nodes do not need to occupy one contiguous block of memory.

3.1 The basic node model

The Week III slides describe a node as having two conceptual parts: a data field and an address field. The address field stores where an adjacent node can be found.

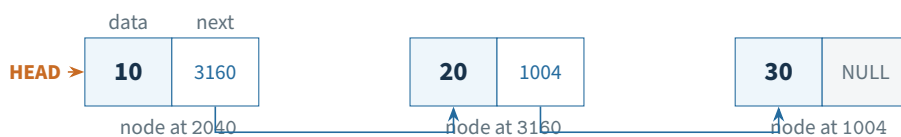


Figure 3.1. Anatomy of a singly linked list. Each node is split into a data field and a next field holding the address of its successor. Note the addresses: the nodes are scattered across memory, and the list holds together only because each one records where the next lives.

The **head** identifies the first node. From the head, the remaining nodes are reached by repeatedly following the stored links.

Key idea

An array answers “where is item *i*?” by index arithmetic. A linked list answers it by following a chain of pointers from the head.

3.2 Why use a linked list?

The Week III slides give four advantages.

Advantage	Meaning
Dynamic size	Memory can be allocated when a new node is needed.
Easy insertion/deletion	Nodes can be added or removed by changing pointers rather than shifting a whole block of elements.
Flexible memory use	Nodes do not need to occupy contiguous addresses.
Useful for ADTs	Linked nodes can support stacks, queues, and graph representations later in the course.

3.3 Limitations of linked lists

The slide deck also emphasizes three disadvantages:

- **No random access.** Reaching a node normally starts from the head.
- **Search is slower than direct array access.** A sequential search takes $O(n)$ in the stated model.
- **Extra memory is required.** Every node stores one or more pointer fields.

Array: A[3] → direct access

Linked list: Head → Node1 → Node2 → Node3

3.4 Singly linked lists

A singly linked list node contains a data field and a `next` pointer. From a given node, the list can move forward to the next node, but there is no direct link back to the previous one.

```
struct Node {
    int data;
    Node* next;
};
```

The slide deck names three basic operations: insertion, deletion, and traversal.

3.4.1 Traversal

```
void display(Node* head) {
    Node* current = head;
    while (current != nullptr) {
        cout << current->data << ' ';
        current = current->next;
    }
    cout << '\n';
}
```

Every visited node advances the pointer one link. If the desired element is near the end, most of the list may need to be visited.

3.4.2 Insert at the beginning

Step 1: Allocate a new node.

Step 2: Point the new node's next to the current head.

Step 3: Move head to the new node.

```
void pushFront(Node*& head, int value) {
    Node* n = new Node{value, head};
    head = n;
}
```

Figure 3.2 shows why the order of those two assignments matters. The new node must be told about the old first node *before* head is moved. Reversing the steps overwrites the only reference to the rest of the list.

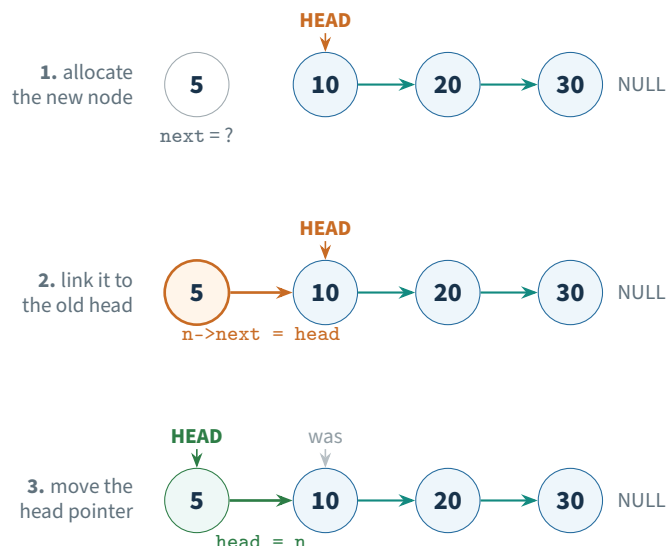


Figure 3.2. Inserting 5 at the front. Step 2 must happen before step 3: once head has been reassigned, nothing else points at node 10, and the remainder of the list is unreachable.

3.4.3 Delete by value

The Week III lab asks for deletion by value. A singly linked list must preserve access to the previous node so its next link can bypass the deleted node.

Figure 3.3 shows the reason. Deleting node 20 means node 10 must be made to point at node 30 — but from node 20 there is no way back to node 10. The traversal must therefore stop one node *early*, which is why the loop above tests `current->next->data` rather than `current->data`.

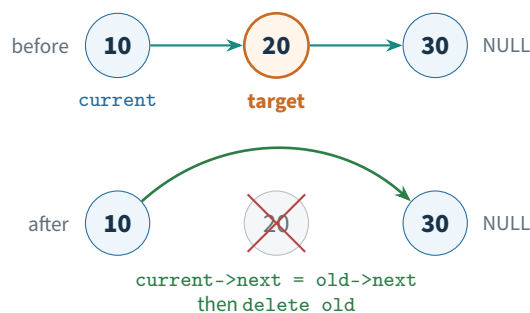


Figure 3.3. Deleting the node holding 20. The predecessor is redirected past the target, and only then is the target freed. Deleting before relinking would leave `current->next` pointing into released memory.

```
bool deleteValue(Node*& head, int value) {
    if (head == nullptr) return false;
```

```

if (head->data == value) {
    Node* old = head;
    head = head->next;
    delete old;
    return true;
}

Node* current = head;
while (current->next != nullptr &&
       current->next->data != value) {
    current = current->next;
}

if (current->next == nullptr) return false;

Node* old = current->next;
current->next = old->next;
delete old;
return true;
}

```

3.5 Doubly linked lists

A doubly linked list stores two links in every node: `prev` and `next`. This supports both forward and backward traversal.

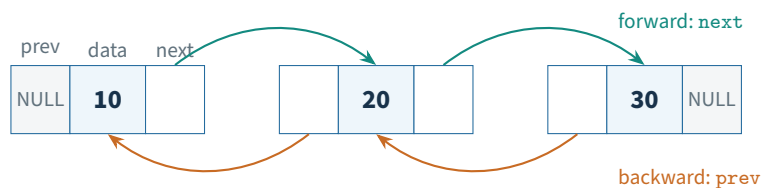


Figure 3.4. A doubly linked list. Every node carries a backward link as well as a forward one, so the list can be walked from either end and a node can be removed without first locating its predecessor. The price is one extra pointer per node.

```

struct DNode {
    int data;
    DNode* prev;
    DNode* next;
};

```

3.6 Singly vs doubly linked lists

Feature	Singly linked list	Doubly linked list
Pointers per node	One: next	Two: prev, next
Direction	Forward only	Forward and backward
Memory use	Lower	Higher
Deletion	Often needs previous-node tracking	Easier when the target node is already known
Complexity of representation	Simpler	More flexible

3.7 Array vs linked list

Feature	Array	Linked list
Memory layout	Contiguous	Non-contiguous allowed
Size	Fixed for a raw array or resized manually	Dynamic through node allocation
Access	Direct using index	Sequential from head
Insertion/deletion	Can require shifting elements	Pointer updates can avoid shifting
Extra memory	No per-element pointer overhead	Pointer fields are stored in each node

Common mistake

“Insertion is easy in a linked list” does not mean “finding the insertion position is always fast.” If the program must first search through n nodes to locate the position, that search still costs traversal time.

Check your understanding

Why can a doubly linked list move backward while a singly linked list cannot? If you already have a pointer to a middle node, which representation stores more information that can help with deletion?

Chapter recap

- A linked list is made of nodes connected by pointers.
- The first node is identified by the head pointer.
- Singly linked lists support forward traversal through `next`.
- Doubly linked lists add `prev` and can traverse in both directions.
- Linked lists are dynamic and flexible but require pointer memory and sequential access.
- Common operations are insertion, traversal, and deletion.

PRACTICAL MILESTONE

Lab 3 of 11

Singly and doubly linked lists

Source alignment — Official Lab Manual — Lab III

Objective. Become familiar with linked lists and gain the ability to use them as a basis for more advanced structures.

Outcomes. Understand linked-list types and develop them from scratch.

Problem directions. Singly-list create/display/delete; doubly-list creation and key search; insertion and deletion at various positions.

Implement both pointer structures

Coverage. Chapter 3.

Source files. Lab03/Task01.cpp (singly linked) and Task02.cpp (doubly linked).

Goal. Implement singly and doubly linked lists, and verify the pointer invariants after every insertion and deletion.

Part A — Singly list. Create, append/insert, display, search, and delete by value.

Part B — Doubly list. Create, display forward/backward, search, insert at multiple positions, and delete by value.

```
struct DNode {
    int data;
    DNode* prev;
    DNode* next;
};

void append(DNode*& head, DNode*& tail, int value) {
    DNode* n = new DNode{value, tail, nullptr};
    if (tail == nullptr) head = tail = n;
    else {
        tail->next = n;
        tail = n;
    }
}
```

Try it yourself

Test insertion and deletion at the head, tail, and a middle position. For the doubly linked list, verify both `next` and `prev` directions after every change.

Verification checklist

- Empty-list and one-node cases do not dereference null pointers.
- Deleting the head updates `head`; deleting the tail updates `tail` where used.
- Doubly linked nodes satisfy `x->next->prev == x` whenever both nodes exist.
- Memory for deleted nodes is released.
- Search reports both found and not-found cases.

Lab deliverable

Submit both implementations plus one pointer diagram before and after a deletion in each list type.

Linked Lists II: Circular Linked Lists

Chapter at a glance

Before you start: Understand the node/head model and singly linked-list traversal from Chapter 3.
By the end, you can: explain the circular link, compare singly/doubly/circular lists, trace circular traversal, and implement beginning/end insertion and deletion plus deletion by value.

A circular linked list changes one crucial link: the last node points back to the first node instead of pointing to NULL. The result is a sequence that can continue in a loop.

4.1 The circular structure

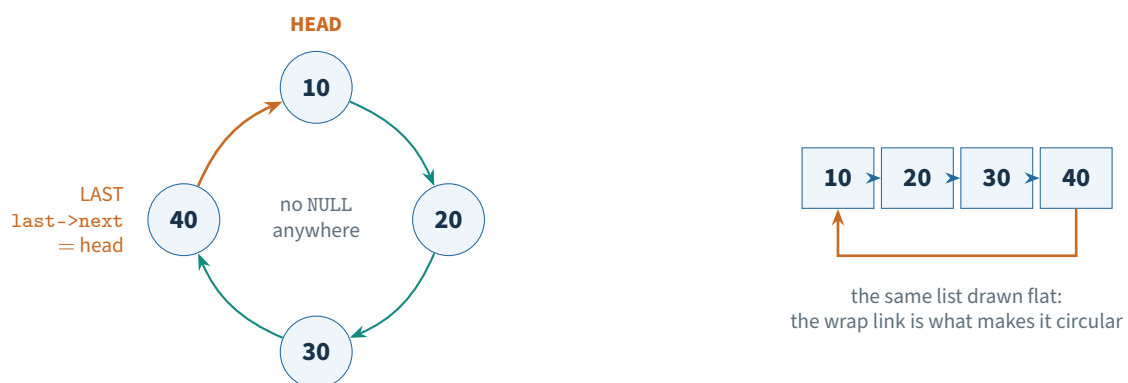


Figure 4.1. A circular linked list drawn as the ring it actually is. The single change from Figure 3.1 is the last node's next field: instead of NULL it stores the address of the head, so there is no end to stop at.

The Week IV slides emphasize four consequences:

- the last node points back to the head;
- there is no NULL at the end;
- traversal can continue indefinitely around the loop;
- therefore traversal needs an explicit stopping condition.

4.2 Traversal: stop when you return to the head

A normal singly linked list often uses `current != nullptr` as the loop condition. That condition does not work for a non-empty circular list because the pointer never becomes null.

```
void display(Node* head) {
    if (head == nullptr) return;

    Node* current = head;
    do {
        cout << current->data << ' ';
        current = current->next;
    } while (current != head);
}
```

```

    current = current->next;
} while (current != head);

cout << '\n';
}

```

Key idea

For a non-empty circular list, the natural traversal invariant is “keep moving until the pointer returns to the starting node.”

That invariant explains the unusual loop shape. The test `current != head` is *already false* on the first iteration, because `current` starts at the head. A plain `while` loop would therefore print nothing at all. The `do-while` form checks the condition only after the body has run once, which is exactly what a ring needs (Figure 4.2).

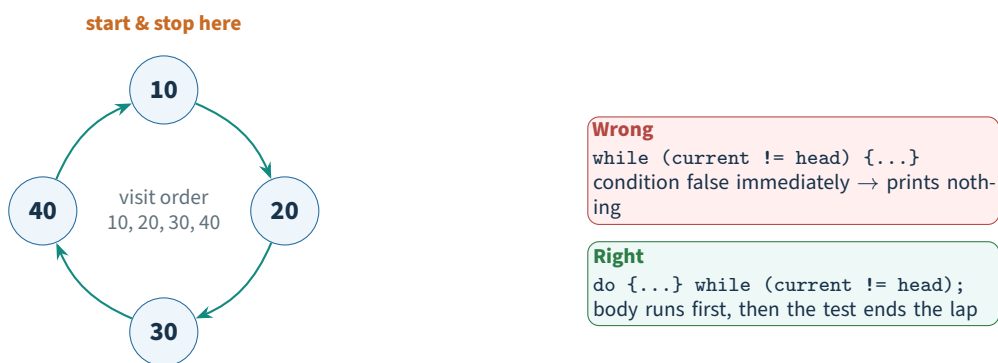


Figure 4.2. Why circular traversal uses `do-while`. The pointer starts on the head and returns to it, so the stopping condition is true at both the beginning and the end of the walk; only the second occurrence should end the loop.

4.3 Singly vs doubly vs circular

Feature	Singly	Doubly	Circular
Last node points to	NULL	NULL	head
Direction	Forward	Forward and backward	Usually forward circular in this course
Pointers per node	next	prev, next	next
Traversal stop	NULL	NULL	back at head
Main use in slides	Simple dynamic list	Two-way navigation	Repeated/cyclic processing

4.4 Insertion at the beginning

Assume `last` points to the final node. Then `last->next` is the head. This representation makes both the tail and head easy to identify.

Step 1: Allocate the new node.

Step 2: If the list is empty, point the node to itself and make it `last`.

Step 3: Otherwise, point the new node to the current head.

Step 4: Set last->next to the new node, making it the new head.

```
void insertBeginning(Node*& last, int value) {
    Node* n = new Node(value, nullptr);

    if (last == nullptr) {
        n->next = n;
        last = n;
        return;
    }

    n->next = last->next;
    last->next = n;
}
```

4.5 Insertion at the end

With the same last representation, insert at the beginning of the circle and then move last to the new node.

```
void insertEnd(Node*& last, int value) {
    Node* n = new Node(value, nullptr);

    if (last == nullptr) {
        n->next = n;
        last = n;
        return;
    }

    n->next = last->next; // new node points to head
    last->next = n;       // old last points to new node
    last = n;            // new node becomes last
}
```

4.6 Deleting from the beginning

There are three cases: empty list, one-node list, and a list with at least two nodes.

```
bool deleteBeginning(Node*& last) {
    if (last == nullptr) return false;

    Node* head = last->next;
    if (head == last) {
        delete head;
        last = nullptr;
        return true;
    }

    last->next = head->next;
    delete head;
    return true;
}
```

4.7 Deleting from the end

A singly circular list has no direct previous pointer, so deletion of the last node requires locating the node whose next is last.

```
bool deleteEnd(Node*& last) {
    if (last == nullptr) return false;
```

```

Node* head = last->next;
if (head == last) {
    delete last;
    last = nullptr;
    return true;
}

Node* current = head;
while (current->next != last) {
    current = current->next;
}

current->next = head;
delete last;
last = current;
return true;
}

```

4.8 Deletion by value

The Week IV lab also requires deletion by value. Traversal must both detect the target and recognize when the full circle has been checked.

```

bool deleteValue(Node*& last, int value) {
    if (last == nullptr) return false;

    Node* prev = last;
    Node* current = last->next; // head

    do {
        if (current->data == value) {
            if (current == prev) { // one-node list
                delete current;
                last = nullptr;
            } else {
                prev->next = current->next;
                if (current == last) last = prev;
                delete current;
            }
            return true;
        }
        prev = current;
        current = current->next;
    } while (current != last->next);

    return false;
}

```

Common mistake

The easiest circular-list bug is an infinite traversal. Every loop over the list must make progress and must have a condition tied to returning to the starting node.

Check your understanding

Why does `while (current != nullptr)` fail as a normal stopping condition in a circular list? What special pointer updates are needed when the list contains only one node?

Chapter recap

- A circular linked list connects the last node back to the head.
- The list does not terminate with NULL.
- Traversal must stop when the pointer returns to the start.

- Insertions and deletions require careful maintenance of the closing link.
- Circular linked lists are useful for repeated or cyclic processing.

PRACTICAL MILESTONE

Lab 4 of 11

Circular linked-list operations

Source alignment — Official Lab Manual — Lab IV

Objective. Study linked-list types more deeply and implement algorithms for circular linked lists.

Outcomes. Understand circular linked lists and implement one from scratch.

Problem directions. Implement the different insertion/deletion operations possible in a circular linked list; compare singly, doubly, and circular lists.

Maintain a circular list without breaking the loop

Coverage. Chapter 4.

Source files. Lab04/Task01.cpp (circular doubly linked) and Task02.cpp (plain doubly linked, for comparison).

Goal. Maintain a circular list through every insertion and deletion without breaking the ring. Implement insertion at beginning/end, deletion from beginning/end, deletion by value, and display.

```
void displayCircular(Node* last) {  
    if (last == nullptr) return;  
    Node* head = last->next;  
    Node* p = head;  
    do {  
        cout << p->data << ' ';  
        p = p->next;  
    } while (p != head);  
}
```

Try it yourself

Use the same input sequence in singly, doubly, and circular implementations. Compare traversal stop conditions and the pointer fields required per node.

Verification checklist

- Empty, one-node, and multi-node cases are tested.
- After every mutation, the last node still points to the head.
- Traversal stops when it returns to the starting node rather than waiting for nullptr.
- Deletion-by-value handles the head, a middle node, and the last node.

Lab deliverable

Submit the required operations, test output, and a three-column comparison of singly, doubly, and circular linked lists.

Stacks

Chapter at a glance

Before you start: Linked lists can be used to build other abstract data types; arrays provide a fixed contiguous alternative.

By the end, you can: explain a stack as a LIFO ADT, trace push/pop/peek operations, identify overflow and underflow, and implement array-based and linked-list stacks in C++.

A **stack** is an abstract data type in which access occurs at one end called the **top**. The defining rule is **LIFO**: Last In, First Out.

5.1 The LIFO principle

If 10 is inserted first, then 20, then 30, the top is 30. Removing an item removes 30 before 20 or 10.

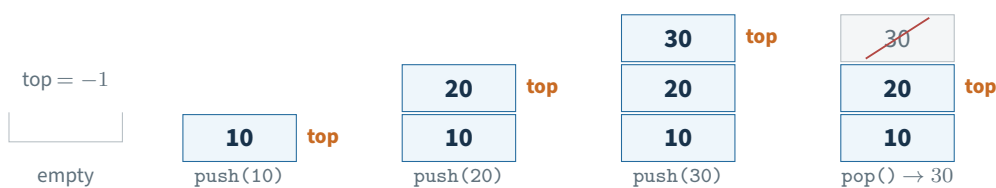


Figure 5.1. A stack under a sequence of operations. Only the top ever changes, and pop always returns the most recent survivor — 30 leaves before 20, which leaves before 10.

The slide deck names the main operations:

push()	Insert a new item at the top.
pop()	Remove the top item.
peek()	Read the top item without removing it.
isEmpty()	Check whether there is no item to access.
isFull()	Check capacity for a fixed-size representation.
display()	Show the stored items.

5.2 Stack states: overflow and underflow

The learning outcomes explicitly ask students to identify both conditions.

Overflow	A push is attempted on a fixed-size stack that has no free slot.
Underflow	A pop or peek is attempted on an empty stack.

Common mistake

Always test the relevant state before changing the top index or dereferencing the top node. Underflow bugs often become invalid memory access.

5.3 Array-based stack

The slides specifically frame the array implementation as fixed-size.

```
class ArrayStack {
private:
    static const int CAPACITY = 5;
    int data[CAPACITY];
    int topIndex = -1;

public:
    bool isEmpty() const { return topIndex == -1; }
    bool isFull() const { return topIndex == CAPACITY - 1; }

    bool push(int value) {
        if (isFull()) return false;
        data[++topIndex] = value;
        return true;
    }

    bool pop(int& removed) {
        if (isEmpty()) return false;
        removed = data[topIndex--];
        return true;
    }

    bool peek(int& value) const {
        if (isEmpty()) return false;
        value = data[topIndex];
        return true;
    }
};
```

The top index is enough to identify the active end of the structure. In the normal implementation above, push, pop, and peek do not scan the existing elements.

5.4 Linked-list stack

A linked-list stack uses the head node as the top. Pushing inserts at the head; popping removes the head.

```
struct StackNode {
    int data;
    StackNode* next;
};

class LinkedStack {
private:
    StackNode* top = nullptr;

public:
    bool isEmpty() const { return top == nullptr; }

    void push(int value) {
        top = new StackNode{value, top};
    }

    bool pop(int& removed) {
        if (isEmpty()) return false;
        StackNode* old = top;
        removed = old->data;
        top = top->next;
        delete old;
        return true;
    }
};
```

```

}

bool peek(int& value) const {
    if (isEmpty()) return false;
    value = top->data;
    return true;
}
};

```

Clarification beyond the slide wording

The Week V lab lists `isFull()` for both array and linked-list stacks. A linked-list stack normally has no predetermined fixed capacity; it is “full” only if an external capacity rule is imposed or memory allocation fails. This distinction is not stated on the slide, so the notes preserve the lab operation while making the implementation assumption explicit.

5.5 Array vs linked-list stack

Feature	Array-based	Linked-list
Capacity in course implementation	Fixed	Dynamic unless a limit is imposed
Top representation	Integer index	Pointer to first node
Push/pop location	End at <code>topIndex</code>	Head node
Overflow	When capacity is exhausted	Not a fixed-size condition by default
Pointer memory	No per-element pointer	One pointer per node

Intuition

A physical stack of plates behaves like the ADT: adding a plate changes only the top, and removing a plate takes the most recently added plate first.

The same discipline runs underneath every C++ program. Each function call pushes a frame holding that call’s parameters and local variables; each `return` pops one (Figure 5.2). This is why a runaway recursion produces a *stack overflow*, and why the recursive tree traversals in Chapter 7 return to the correct parent without storing one explicitly.

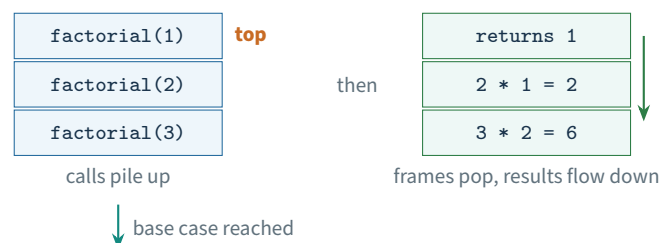


Figure 5.2. The call stack during `factorial(3)`. Calls pile up until the base case is reached, then unwind in reverse order, each frame resuming exactly where it paused. The stack is what remembers the half-finished multiplications.

5.6 Lab-manual application: infix to postfix

The official lab manual adds one stack problem that is not spelled out in the Week V slide deck: **conversion of an infix expression to postfix using a stack**. The stack temporarily stores operators while operands are sent directly to the output.

For example, the infix expression

$$A + B * C$$

becomes

$$A B C * +.$$

Multiplication is emitted before addition because it has higher precedence.

Step 1: Read the expression from left to right.

Step 2: Send operands directly to the postfix output.

Step 3: Push an opening parenthesis onto the operator stack.

Step 4: For an operator, pop operators of higher precedence (and, for left-associative operators, equal precedence) before pushing the new operator.

Step 5: On a closing parenthesis, pop until the matching opening parenthesis is removed.

Step 6: At the end, pop all remaining operators.

Clarification beyond the slide wording

The lab manual names infix-to-postfix conversion as practice but does not provide the precedence/associativity algorithm in the supplied material. The procedure above is standard supporting context so the named lab task can be implemented.

Figure 5.3 traces the whole conversion for a slightly larger expression. Read it as a table of states: the stack column is the operator stack from bottom to top, and the output column only ever grows.

Read	Action	Stack	Output
A	operand: emit		A
+	stack empty: push	+	A
B	operand: emit	+	A B
*	higher than +: push	+ *	A B
C	operand: emit	+ *	A B C
-	pop * then +, push -	-	A B C * +
D	operand: emit	-	A B C * + D
end	pop remaining		A B C * + D -

Figure 5.3. Converting $A + B * C - D$ to postfix. When $-$ arrives, the stack holds $+$ and $*$; both have precedence at least as high as $-$, so both are emitted before $-$ is pushed. That single rule is what encodes operator precedence.

Check your understanding

After `push(10)`, `push(20)`, `push(30)`, `pop()`, and `peek()`, what value is returned by `peek()`? What condition causes underflow?

Chapter recap

- A stack is an ADT with access at the top.
- LIFO means the most recently inserted item is removed first.
- `push()` inserts, `pop()` removes, and `peek()` reads the top.
- Fixed-size stacks can overflow; empty stacks can underflow.
- Stacks can be implemented with arrays or linked lists.

PRACTICAL MILESTONE

Lab 5 of 11

Stack ADT and expression processing

Source alignment — Official Lab Manual — Lab V

Objective. Build intuition for stacks, their operations, implementations, and use cases.

Outcomes. Understand stack behavior; use push/pop/peek; apply a stack to unseen programming problems.

Problem directions. Implement a stack with an array; implement a stack with a linked list; convert infix notation to postfix using a stack.

Implement the stack ADT twice

Coverage. Chapter 5.

Source files. Lab05/Task03.cpp (array stack), Task04.cpp (linked stack), and Task05.cpp (infix to postfix).

Goal. Implement the stack ADT with an array and with a linked list, then apply a stack to infix-to-postfix conversion.

Required operations are push(), pop(), peek(), isFull(), isEmpty(), and display() for the array version and the corresponding linked-list behavior.

```
// Suggested trace
push(10);
push(20);
push(30);
peek();      // 30
pop();       // removes 30
display();   // 20 10 from top to bottom
```

Try it yourself

Deliberately trigger array overflow and stack underflow. Then implement the manual's infix-to-postfix exercise and test $A+B*C$ and $(A+B)*C$.

Clarification beyond the slide wording

A linked-list stack normally has no predetermined fixed capacity. If your interface contains isFull(), document whether you impose a maximum node count; otherwise only allocation failure makes the structure unable to grow.

Verification checklist

- LIFO order is visible in the output.
- Underflow is handled before reading/removing the top.
- Array overflow is handled before writing past capacity.
- The infix-to-postfix result respects parentheses and operator precedence.

Lab deliverable

Submit both stack implementations, one overflow/underflow trace, and the infix-to-postfix converter with at least two test expressions.

Queues

Chapter at a glance

Before you start: Know the constrained-access idea from stacks and the array/linked-list implementation alternatives.

By the end, you can: explain FIFO, trace front/rear movement, implement enqueue/dequeue, explain why circular queues reuse array space, and distinguish linear, circular, and priority queues at the level covered by the slides.

A **queue** is a linear data structure that follows **FIFO**: First In, First Out. Insertion occurs at the **rear**; deletion occurs at the **front**.

6.1 Front and rear



Figure 6.1. A queue has two active ends. Items join at the rear and leave from the front, so the order of departure matches the order of arrival — the opposite discipline to a stack.

The Week VI slides use the terms **Create Q**, **Enqueue(Q,item)**, and **Dequeue(Q)**. The enqueue operation is stated as $O(1)$ in the slide implementation model.

6.2 Core queue operations

Create Q	Create an empty queue.
enqueue()	Add an item at the rear.
dequeue()	Remove and return the oldest item at the front.
peek/front	Read the front item without removing it.
isEmpty()	Check whether the queue contains no items.
isFull()	Check capacity in a fixed array implementation.

6.3 A simple linear array queue

A basic fixed array can use indexes for front and rear.

```
class LinearQueue {
private:
    static const int CAP = 5;
    int data[CAP];
    int front = 0;
    int rear = -1;
```

```

public:
    bool isEmpty() const { return rear < front; }
    bool isFull() const { return rear == CAP - 1; }

    bool enqueue(int value) {
        if (isFull()) return false;
        data[++rear] = value;
        return true;
    }

    bool dequeue(int& value) {
        if (isEmpty()) return false;
        value = data[front++];
        return true;
    }
};

```

This model exposes the limitation described by the slides: after repeated dequeues, unused positions may exist at the beginning even though `rear` has reached the end of the array.

Figure 6.2 shows the failure. Both `front` and `rear` only ever move right, so the queue drifts up the array and eventually reports itself full while sitting on empty slots. Shifting everything back down would fix it, but that costs $O(n)$ on every dequeue and throws away the whole point of the structure.

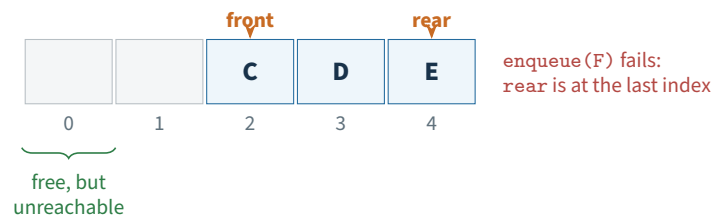


Figure 6.2. The “false full” problem in a linear array queue. After three enqueues and two dequeues, `rear` has reached the last index and further enqueues are rejected — even though indices 0 and 1 are free. The array is not full; the queue has merely walked off the end of it.

6.4 Circular queue: reusing array space

The circular queue addresses that limitation by connecting the logical last position back to the first. The slides call this a **ring buffer** and state that modular arithmetic is used for enqueue and dequeue.

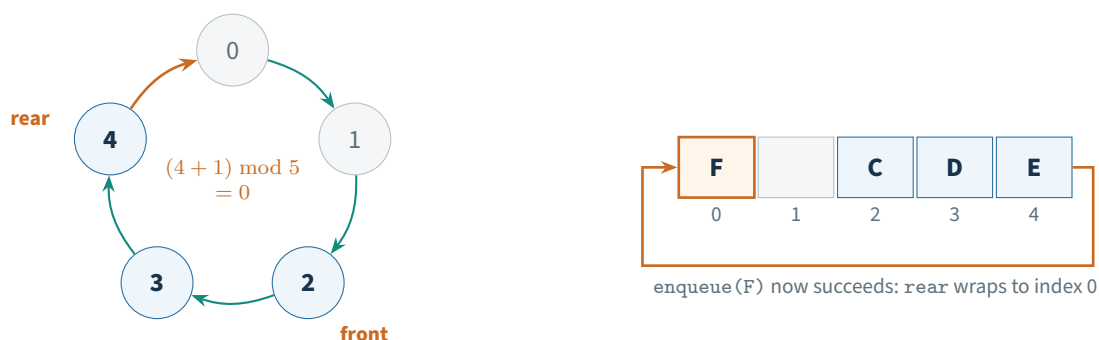


Figure 6.3. The same five slots viewed as a ring. Advancing with $(\text{index} + 1) \bmod N$ sends `rear` from index 4 back to index 0, so the free space at the front of the array is reclaimed instead of stranded.

For capacity N , moving to the next slot uses

$$\text{next} = (\text{index} + 1) \bmod N.$$

Worked example

With capacity 5, if the rear is at index 4, then the next rear position is

$$(4 + 1) \bmod 5 = 0.$$

The index wraps back to the beginning instead of moving outside the array.

6.5 Circular queue implementation pattern

One robust representation stores the number of active elements in addition to front and rear.

```
class CircularQueue {
private:
    static const int CAP = 5;
    int data[CAP];
    int front = 0;
    int rear = 0;
    int count = 0;

public:
    bool isEmpty() const { return count == 0; }
    bool isFull() const { return count == CAP; }

    bool enqueue(int value) {
        if (isFull()) return false;
        data[rear] = value;
        rear = (rear + 1) % CAP;
        ++count;
        return true;
    }

    bool dequeue(int& value) {
        if (isEmpty()) return false;
        value = data[front];
        front = (front + 1) % CAP;
        --count;
        return true;
    }
};
```

6.6 Linked-list queue

A linked queue normally keeps both a front pointer and a rear pointer so enqueue can add at the rear without traversing the list.

```
struct QNode {
    int data;
    QNode* next;
};

class LinkedQueue {
private:
    QNode* front = nullptr;
    QNode* rear = nullptr;

public:
    bool isEmpty() const { return front == nullptr; }

    void enqueue(int value) {
        QNode* n = new QNode{value, nullptr};
        if (rear == nullptr) {
            front = rear = n;
        } else {
            rear->next = n;
            rear = n;
        }
    }
};
```

```

    }

    bool dequeue(int& value) {
        if (isEmpty()) return false;
        QNode* old = front;
        value = old->data;
        front = front->next;
        if (front == nullptr) rear = nullptr;
        delete old;
        return true;
    }
};

```

6.7 Linear, circular, and priority queues

The lecture deck asks students to differentiate these queue types, and the official lab manual adds the processing rules for a **priority queue**.

Queue type	Ordering rule	Course role
Linear queue	FIFO	Front removes oldest; rear inserts newest.
Circular queue	FIFO with wrap-around array positions	Reuses vacant positions with modulo arithmetic.
Priority queue	Higher priority is processed before lower priority; equal-priority items follow arrival order	Models work where urgency matters more than pure arrival time.

The lab manual states two priority rules:

1. an element of higher priority is processed before an element of lower priority;
2. when two elements have the same priority, they are processed in the order in which they were added.

It also states that priority queues can be implemented using arrays or linked lists.

Source alignment — Queue lab manual

The manual supplies the priority-ordering rules and the array/linked-list implementation options, but it does not specify a particular insertion strategy, heap representation, or priority convention such as whether a numerically larger value means higher priority. Any implementation should document that design choice.

Check your understanding

Why can a simple linear array queue appear full even when unused slots exist near index 0? How does modulo arithmetic fix that issue in a circular queue?

Chapter recap

- A queue follows FIFO.
- Enqueue inserts at the rear; dequeue removes at the front.
- Queues can be implemented with arrays or linked lists.
- A circular queue connects the logical last position to the first and uses modulo arithmetic to reuse space.

- Priority queues use priority rather than pure arrival order.

PRACTICAL MILESTONE

Lab 6 of 11

Linear, linked, circular, and priority queues

Source alignment — Official Lab Manual — Lab VI

Objective. Build proper intuition for queues and implement different queue types.

Outcomes. Understand queue variants and efficient array/linked-list implementations.

Problem directions. Array queue with enqueue/dequeue; linked-list queue with enqueue/dequeue; circular-array queue problems. The manual also defines priority-queue processing rules.

Implement queues and demonstrate wrap-around

Coverage. Chapter 6.

Source files. Lab05/Task01.cpp (circular array queue), Task02.cpp (linked queue), and Task06.cpp (priority queue).

Goal. Implement array and linked queues, then demonstrate circular wrap-around without confusing the full and empty states.

Build an array queue and a linked-list queue. Then implement a circular queue where index movement uses modulo arithmetic.

```
// Circular queue, capacity 5
// enqueue 10, 20, 30, 40
// dequeue twice
// enqueue 50, 60
// rear must wrap without breaking FIFO order
```

Try it yourself

Print front, rear, and either count or your chosen full/empty state after every circular operation. As an extension, represent items as (value, priority) and implement the two priority rules from the lab manual.

Verification checklist

- Array and linked-list queues preserve FIFO order.
- Dequeuing the final linked node resets both front and rear correctly.
- Circular wrap-around visibly reuses low array indexes.
- Full and empty circular states cannot be confused.
- If a priority queue is implemented, equal-priority items preserve arrival order.

Lab deliverable

Submit array and linked-list queue implementations plus a circular-queue trace that reuses earlier array slots. Include the priority-queue extension if assigned by your instructor.

Stage 3

Trees and Hierarchies

Trees I: Terminology, Traversal, and Binary Trees

Chapter at a glance

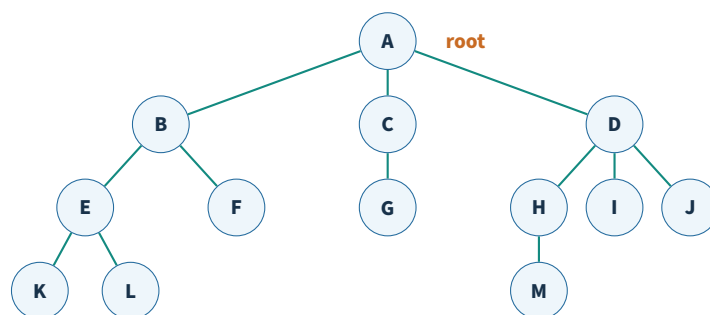
Before you start: Know linear and non-linear classification, linked-node representation, and basic pointer-based traversal.

By the end, you can: define a tree and subtree, use root/parent/sibling/level/height terminology, trace inorder/preorder/postorder traversal, and explain array and linked representations of a binary tree.

A **tree** is a non-linear data structure in which data is stored hierarchically. The Trees I deck defines a tree recursively: there is a specially designated **root**, and the remaining nodes can be partitioned into disjoint subtrees of that root.

7.1 Root and subtrees

The slide example uses root *A* with the following structure.



The Trees I slide explicitly identifies the subtree rooted at *B* as one subtree of *A*, and similarly identifies the *C* and *D* branches as other subtrees.

7.2 Tree terminology

7.2.1 Edge

An **edge** is the connecting link between two nodes.

Clarification beyond the slide wording

The slide says that with N nodes there will be a *maximum* of $N - 1$ edges. For an ordinary connected tree with N nodes, the standard invariant is stronger: it has *exactly* $N - 1$ edges. The slide wording is preserved here and the standard property is supplied as clarification.

7.2.2 Parent and sibling

If node N is the immediate predecessor of node X , then N is the parent of X . Nodes with the same parent are siblings. In the slide example, A is a parent of B , C , and D ; E and F are siblings.

7.2.3 Internal node

An internal node has at least one child. In the diagram above, examples include A , B , C , D , E , and H .

Clarification beyond the slide wording

One slide lists $A, B, C, D, E, F, G, H, I, J$ as internal nodes, but the accompanying diagram shows several of those nodes without children. These notes use the slide's own definition — “has at least one child” — when classifying the drawn example.

7.2.4 Level

The **level** of a node is the number of edges on the path from that node to the root. The root is at level 0. If a node is at level ℓ , its children are at level $\ell + 1$.

7.2.5 Height

The slides define the height of a node as the number of edges on the longest path from that node down to a leaf. In the example, H has one child M , so its height is 1.

Worked example

Using the recreated slide tree: A is at level 0, D is at level 1, H is at level 2, and M is at level 3. The height of H is 1 because the longest downward path is $H \rightarrow M$.

7.3 Tree traversal

Traversal means visiting every node exactly once. The Trees I deck teaches three depth-first traversal orders.

Traversal	Rule
Inorder	Left subtree $\rightarrow$ root $\rightarrow$ right subtree
Preorder	Root $\rightarrow$ left subtree $\rightarrow$ right subtree
Postorder	Left subtree $\rightarrow$ right subtree $\rightarrow$ root

These traversal rules are most naturally applied to binary trees, because each node has a clear left and right subtree.

The three orders differ in one respect only: *when the root is visited* relative to its subtrees. Figure 7.1 applies all three to the same tree, numbering each node with its position in the output sequence.

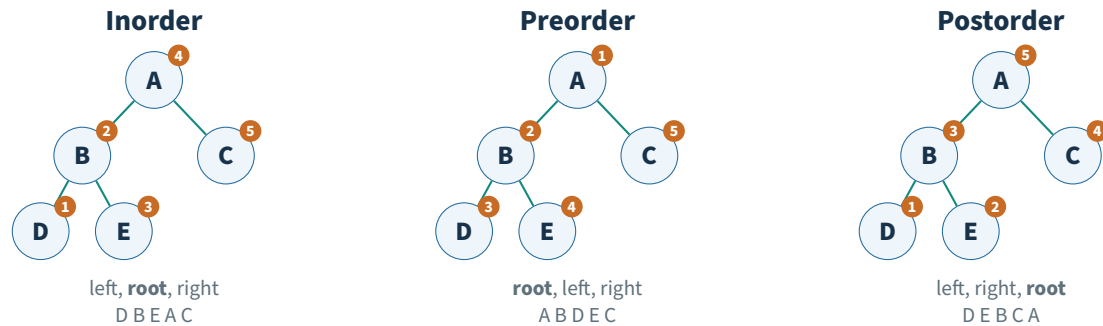


Figure 7.1. The same binary tree under all three depth-first traversals. The shape of the walk is identical in every case — down the left, down the right, back up — and only the moment at which a node is *recorded* moves. Inorder on a search tree yields sorted output, which is why it is the one used most often.

7.4 Binary trees

A **binary tree** is a tree in which each node has at most two children. A binary-tree node can be represented with two pointers.

```
struct TreeNode {
    int data;
    TreeNode* left;
    TreeNode* right;
};
```

7.4.1 Recursive traversal code

```
void inorder(TreeNode* root) {
    if (root == nullptr) return;
    inorder(root->left);
    cout << root->data << ' ';
    inorder(root->right);
}

void preorder(TreeNode* root) {
    if (root == nullptr) return;
    cout << root->data << ' ';
    preorder(root->left);
    preorder(root->right);
}

void postorder(TreeNode* root) {
    if (root == nullptr) return;
    postorder(root->left);
    postorder(root->right);
    cout << root->data << ' ';
}
```

7.5 Array representation of a binary tree

The Trees I slide states that if a root is stored at array index I , then the left child is at $2I$ and the right child at $2I + 1$.

Clarification beyond the slide wording

Those formulas correspond to a **1-based** array representation. C++ raw arrays are usually indexed from 0. With a zero-based representation, the common formulas are

$$\text{left}(i) = 2i + 1, \quad \text{right}(i) = 2i + 2,$$

and the parent of a non-root node is $\lfloor (i - 1) / 2 \rfloor$.

Worked example

Using zero-based indexing, node at index 2 has left child at $2(2) + 1 = 5$ and right child at $2(2) + 2 = 6$. Using the slide's 1-based convention, a node at index 2 has children at 4 and 5.

Figure 7.2 shows the mapping directly. Reading the tree level by level, left to right, and writing the values into consecutive array slots is all the representation does — the parent-child links are then recoverable by arithmetic, with no pointers stored at all.

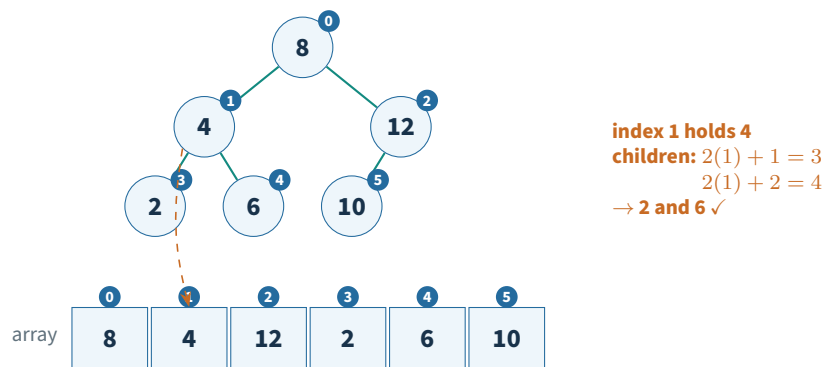
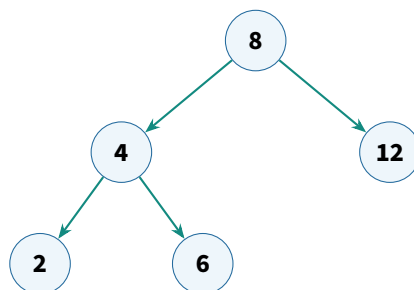


Figure 7.2. A complete binary tree stored in an array, zero-based. Level-order position becomes array index, so the children of index i sit at $2i + 1$ and $2i + 2$. This is the same layout the heap in Chapter 8 relies on.

7.6 Linked representation of a binary tree

A linked representation stores data plus two pointers: one to the left child and one to the right child. A missing child is represented by a null pointer.

**Check your understanding**

State the inorder, preorder, and postorder rules without looking back. What indexing assumption is required for the slide formula $2I$ and $2I + 1$?

Chapter recap

- Trees store data hierarchically and have a designated root.
- Parent, sibling, internal node, level, and height describe structural relationships.
- Inorder, preorder, and postorder visit the root in different positions relative to its subtrees.
- A binary tree has at most two children per node.
- Binary trees can use array or linked representations.

PRACTICAL MILESTONE

Lab 7 of 11

Tree representation and traversal

Source alignment — Official Lab Manual — Lab VII

Objective. Learn the tree ADT and apply trees to real-life problems.

Outcomes. Implement a tree using an array and a linked representation; implement traversal methods; choose an appropriate tree structure.

Problem directions. Tree representation; binary-tree implementation; traversal algorithms; inserting, deleting, and updating tree elements.

Represent the same binary tree two ways

Coverage. Chapter 7.

Source files. Lab06/Task02.cpp — the same tree in linked and array form, with all three traversals.

Goal. Represent one binary tree in both linked and array form and produce all three traversal orders.

```
TreeNode* root = new TreeNode{8, nullptr, nullptr};
root->left = new TreeNode{4, nullptr, nullptr};
root->right = new TreeNode{12, nullptr, nullptr};
root->left->left = new TreeNode{2, nullptr, nullptr};
root->left->right = new TreeNode{6, nullptr, nullptr};
```

Implement inorder, preorder, and postorder traversal. Then represent the same shape in an array and identify each parent/child index.

Try it yourself

Write all three traversal sequences before running the program. Add one update operation and one structural insertion/deletion exercise specified by your lab session.

Verification checklist

- Traversals visit every stored node exactly once.
- The linked and array representations describe the same shape.
- Your array child-index formula matches the chosen 0-based or 1-based convention.
- Empty subtrees terminate recursion safely.

Lab deliverable

Submit both representations, all three traversal outputs, and a brief note stating your indexing convention.

Trees II: BSTs, Heaps, and Height Balancing

Chapter at a glance

Before you start: Understand binary trees, node height, and the array/linked representations from Chapter 7.

By the end, you can: state the BST ordering rule, distinguish almost-complete and heap structures, compare max-heaps and min-heaps, and explain the purpose of height balancing.

The Trees II slides specialize the general binary-tree idea into structures that impose additional rules. Those rules are valuable because they constrain where values can appear.

8.1 Binary search tree (BST)

A **binary search tree** is a binary tree with an ordering restriction. In the slide wording, the root value is larger than any value in its left subtree and smaller than any value in its right subtree. The same ordering idea is applied recursively at each node.

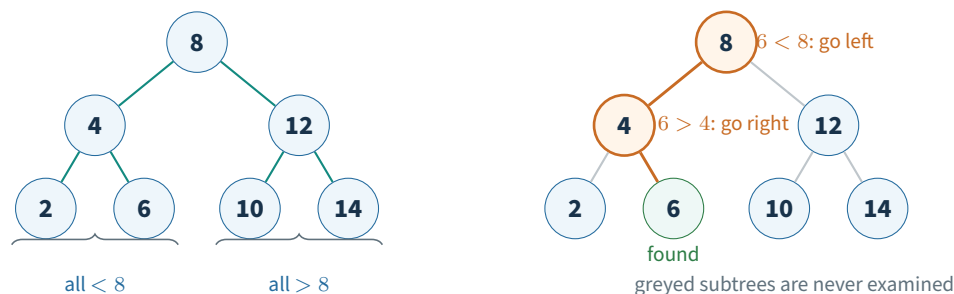


Figure 8.1. The BST ordering rule, and the search path it produces. Every comparison discards an entire subtree, so searching for 6 touches three nodes rather than seven.

Searching compares the target with the current node and chooses one subtree.

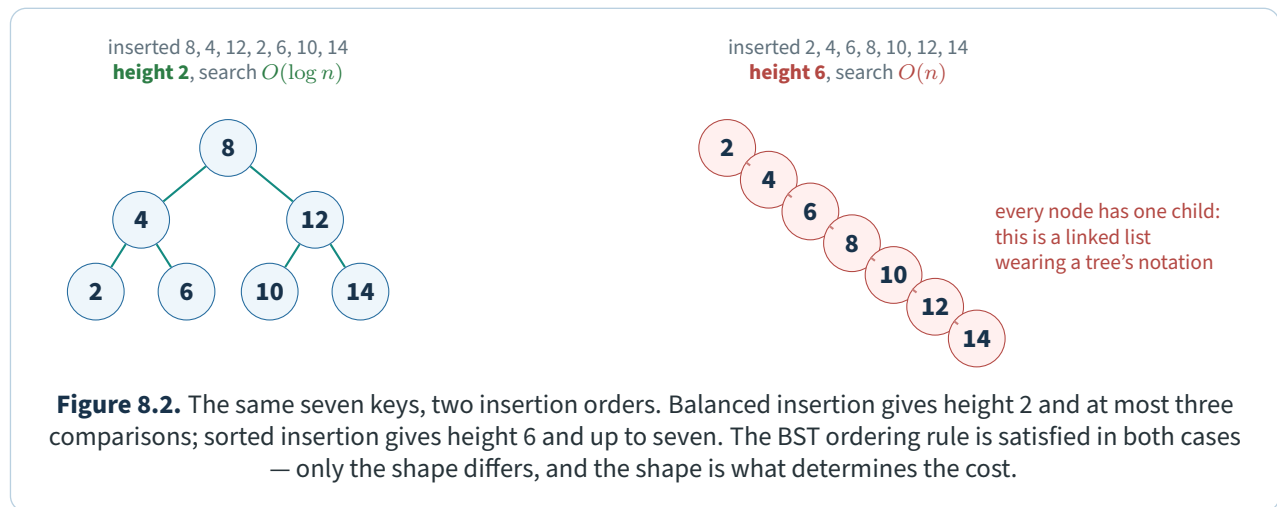
```
bool contains(TreeNode* root, int target) {
    while (root != nullptr) {
        if (target == root->data) return true;
        if (target < root->data) root = root->left;
        else root = root->right;
    }
    return false;
}
```

Clarification beyond the slide wording

The slide states that BST searching is reduced to $O(\log N)$. That is the expected/balanced-tree behavior. A plain BST can become highly unbalanced (for example, inserting already sorted keys), in which case search can degrade to $O(N)$. The later height-balancing concept explains why keeping height controlled matters.

Figure 8.2 makes that failure concrete. Inserting the same seven keys in sorted order produces a tree that is

really a linked list: every node has one child, the height grows with N rather than $\log N$, and the search advantage disappears entirely.



8.2 BST insertion

Insertion follows the same comparisons used by search.

```
TreeNode* insertBST(TreeNode* root, int value) {
    if (root == nullptr) {
        return new TreeNode{value, nullptr, nullptr};
    }

    if (value < root->data)
        root->left = insertBST(root->left, value);
    else if (value > root->data)
        root->right = insertBST(root->right, value);

    return root;
}
```

The slide deck does not specify how duplicate keys should be handled, so the example above simply ignores duplicates.

8.3 BST deletion

Deletion is the one BST operation with genuinely distinct cases, because removing a node must leave the ordering rule intact for everything below it (Figure 8.3).

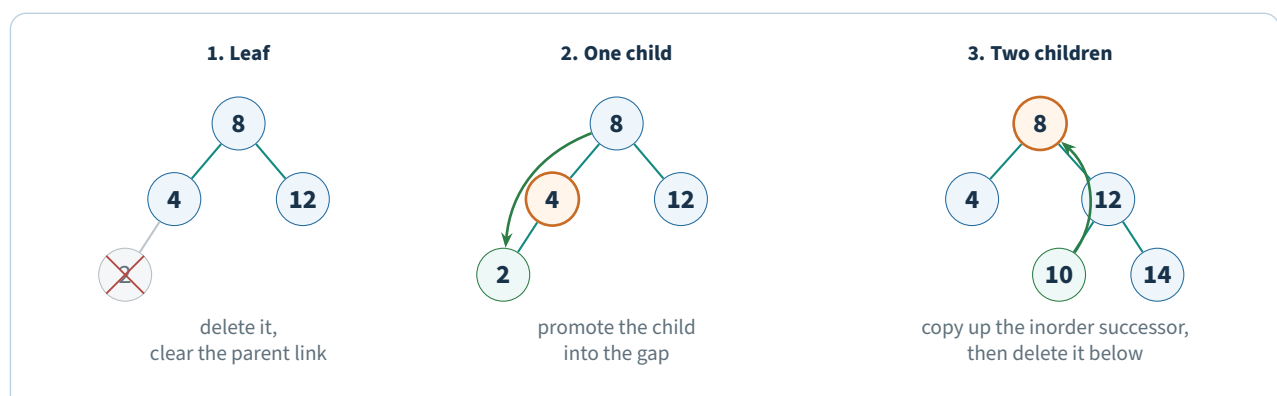


Figure 8.3. The three deletion cases. A leaf simply goes. A node with one child is replaced by that child. A node with two children keeps its position in the tree but takes the value of its *inorder successor* — the smallest key in its right subtree — which is then deleted from below, where it is guaranteed to have at most one child.

Clarification beyond the slide wording

Deletion is not spelled out in the supplied Trees II material. The three-case treatment above is the standard formulation, included so the operation can be implemented for the lab and the final project.

8.4 Almost complete binary tree

The slides define an **almost complete binary tree** as a binary tree where all levels are full except possibly the last, and the last level is filled from the left as far as possible.

This shape is important for heaps because it allows a compact array representation without arbitrary gaps.

8.5 Heaps

A **heap** is a tree-based structure whose shape is a complete/almost-complete binary tree and whose values satisfy a heap-order rule.

Heap type	Rule from the slides
Max-heap	The root key is maximum among its children, and the same property holds recursively in every subtree.
Min-heap	The root key is minimum among its children, and the same property holds recursively in every subtree.

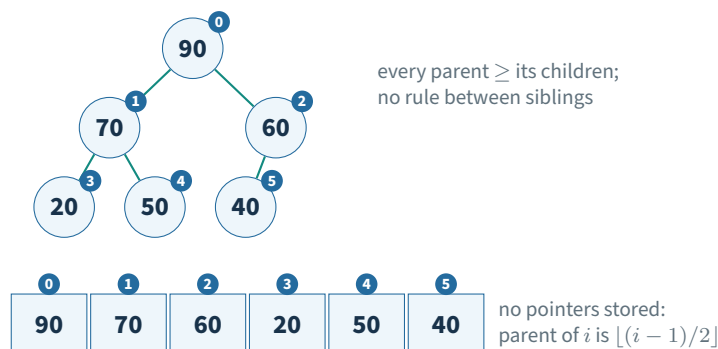


Figure 8.4. A max-heap and its array. The heap-order rule constrains each parent against its own children only — it says nothing about left-to-right order, which is why 50 may sit to the left of 60. Because the shape is almost complete, the tree packs into the array with no gaps.

8.6 Array representation of a heap

Using zero-based indexing, the binary-tree relationships are

$$\text{left}(i) = 2i + 1, \quad \text{right}(i) = 2i + 2, \quad \text{parent}(i) = \left\lfloor \frac{i-1}{2} \right\rfloor.$$

The nearly complete shape makes these formulas practical.

8.7 Insertion into a max-heap

A standard implementation inserts at the next available array position and then moves the new value upward while the heap-order property is violated.

```
void pushMaxHeap(vector<int>& heap, int value) {
    heap.push_back(value);
    int i = static_cast<int>(heap.size()) - 1;

    while (i > 0) {
        int p = (i - 1) / 2;
        if (heap[p] >= heap[i]) break;
        swap(heap[p], heap[i]);
        i = p;
    }
}
```

Clarification beyond the slide wording

The Trees II slides require heap creation plus insertion and deletion in the lab, but they do not spell out heapify-up/heapify-down algorithms. The code here is an implementation expansion of that lab requirement.

Figure 8.5 traces that loop. The new value enters at the only position the shape allows — the next free array slot — and then climbs, swapping with its parent until a larger parent stops it. Because the tree's height is $\log n$, the climb is bounded by $O(\log n)$ swaps.

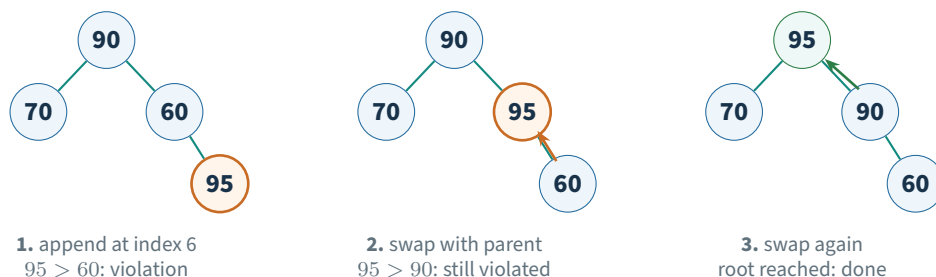
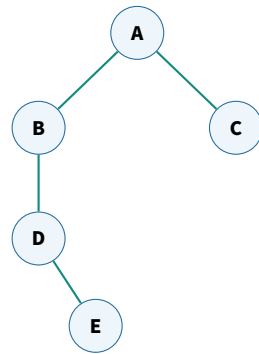


Figure 8.5. Inserting 95 into the max-heap. The value is appended at index 6, then swapped upward past 60 and 90 until the heap-order rule holds again. Only nodes on the path from the insertion point to the root are ever touched.

8.8 Height-balancing trees

The slides define a height-balancing tree by requiring the height difference between a node's left and right subtrees to be no more than 1. If insertion makes that difference exceed 1, an appropriate balancing action is taken immediately.



If one side grows too deep, the tree no longer satisfies the slide's balance condition.

The slide deck names AVL trees and B-trees as examples associated with height balancing and states that the advantage is $O(\log N)$ worst-case searching when height is controlled.

Clarification beyond the slide wording

The supplied Trees II deck names AVL and B-tree concepts but does not teach rotations, B-tree node-order rules, split/merge operations, or full B-tree algorithms. The notes therefore keep those structures at the conceptual level and preserve B-tree creation/add/remove as lab-directed practice rather than inventing a detailed lecture that was not present.

Check your understanding

What additional condition turns a binary tree into a BST? What two independent properties define a heap: its shape and its ordering rule?

Chapter recap

- A BST orders smaller values to the left and larger values to the right.
- BST search is logarithmic when tree height stays logarithmic; an unbalanced plain BST can degrade.
- Almost-complete trees fill levels from left to right.
- Max-heaps keep large values above smaller children; min-heaps do the opposite.
- Height balancing limits left/right height difference and protects search efficiency.

PRACTICAL MILESTONE

Lab 8 of 11

BSTs, heaps, and balancing structures

Source alignment — Official Lab Manual — Lab VIII

Objective. Understand BSTs, heaps, AVL/height balancing, and B-trees and apply those structures to problems.

Outcomes. Learn BST, heap, and B-tree structures and implement/apply them computationally.

Problem directions. Create max/min heaps; insert/delete heap elements; solve BST problems; practice B-tree creation, insertion, and removal.

Compare ordering rules across specialized trees

Coverage. Chapter 8.

Source files. Lab06/Task01.cpp (binary search tree) and Task03.cpp (max-heap and min-heap).

Goal. Compare the ordering rules of binary search trees and heaps, and show how each restores its rule after a change.

```
// BST insertion order
// 50, 30, 70, 20, 40, 60, 80
// inorder should be sorted.

// Max-heap insertion order
// 30, 10, 50, 20, 40
// print the heap array after every insertion.
```

Try it yourself

Insert 10,20,30,40,50 into a plain BST and draw the resulting height. Compare that shape with a balanced tree. Then create both a min-heap and max-heap from the same keys.

How to use this lab

The supplied course sources name B-tree creation/insertion/removal but do not provide the order/minimum-degree rules needed for a unique implementation. Follow the B-tree parameters specified by your instructor rather than inventing them.

Verification checklist

- BST inorder output is sorted for the chosen key policy.
- A max-heap parent is not smaller than either child; a min-heap parent is not larger than either child.
- Heap insertion/deletion restores the heap property.
- Complexity claims are related to tree height, not only node count.

Lab deliverable

Submit a BST program, one heap implementation with insertion/deletion, and instructor-directed B-tree exercises where assigned.

Stage 4

Graphs and Traversal

Graphs

Chapter at a glance

Before you start: Understand non-linear structures, linked representations, and queue/stack behavior.

By the end, you can: define $G(V, E)$, distinguish directed and undirected graphs, represent a graph with an adjacency matrix or adjacency list, trace BFS and DFS, and explain the spanning-tree constraints stated in the slides.

A **graph** generalizes the idea of connected data. The graph $G(V, E)$ contains a set of vertices V and a set of edges E that connect vertices.

9.1 Vertices and edges

The graph example in the slides uses

$$V = \{A, B, C, D, E\}$$

and

$$E = \{(A, B), (B, E), (A, D), (A, C), (C, D), (D, E), (B, D)\}.$$

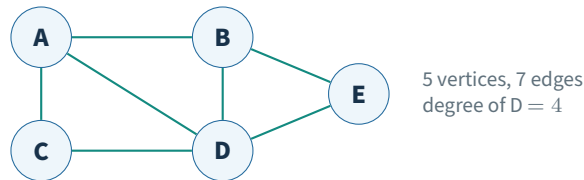


Figure 9.1. The running example graph used throughout this chapter: five vertices and seven undirected edges. Vertex D is adjacent to every other vertex, while C reaches E only by passing through A, B, or D.

The slides contrast trees and graphs by noting that a tree is acyclic while a graph may contain cycles. They also motivate graphs with a city-distance example: cities become vertices and distances become edge information.

9.2 Directed and undirected graphs

Directed graph Edges have direction. (a, b) and (b, a) can represent different relationships.

Undirected graph Edges have no direction. (a, b) and (b, a) are equivalent descriptions of the same connection.

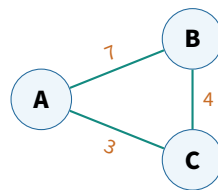


9.3 Weighted and unweighted graphs

The official lab manual extends the graph classification beyond direction.

Weighted graph Each edge carries an associated value such as distance or cost. The manual's examples include distances between cities and communication cost between workstations.

Unweighted graph Edges indicate that two vertices are related, but no numerical edge weight is attached.



The same topology can be weighted by distance, cost, time, etc.

Key idea

Direction and weight are separate properties. A graph can be directed and weighted, directed and unweighted, undirected and weighted, or undirected and unweighted.

9.4 Graph representations

The graph deck teaches two storage strategies:

- sequential or **adjacency matrix** representation;
- linked or **adjacency list** representation.

9.4.1 Adjacency matrix

An adjacency matrix uses a matrix entry A_{ij} to represent the connection or associated edge weight from node i to node j .

For the undirected example above, a simple unweighted 0/1 matrix is

	A	B	C	D	E
A	0	1	1	1	0
B	1	0	0	1	1
C	1	0	0	1	0
D	1	1	1	0	1
E	0	1	0	1	0

Clarification beyond the slide wording

The slide image appears to show a 1 on the D, D diagonal entry, while the edge set printed on the preceding slide does not list a self-loop (D, D) . The matrix above follows the printed edge set. This discrepancy is made explicit rather than silently copied.

Figure 9.2 puts both representations of Figure 9.1 side by side. They hold identical information; they differ in what they make cheap. The matrix answers “is A adjacent to E?” in one lookup but always occupies V^2

cells. The list stores only the edges that exist, so it is far smaller on a sparse graph, but the same question requires scanning A's neighbours.

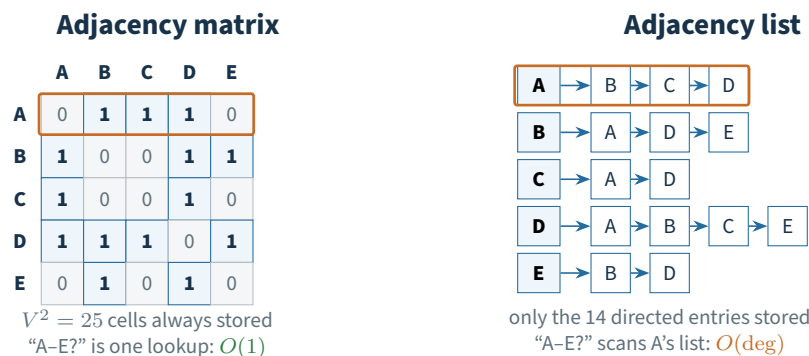


Figure 9.2. The same graph as an adjacency matrix and an adjacency list. Row A of the matrix and the chain hanging off A carry exactly the same fact — that A neighbours B, C, and D — but the matrix also stores the two zeros recording what A is *not* connected to.

9.4.2 Adjacency list

An adjacency list stores, for each vertex, the vertices connected to it. The slides emphasize that it is storage-efficient because values are stored for edges rather than allocating a full matrix.

```
vector<vector<int>>> adj(5);

// A=0, B=1, C=2, D=3, E=4
adj[0] = {1, 2, 3};
adj[1] = {0, 3, 4};
adj[2] = {0, 3};
adj[3] = {0, 1, 2, 4};
adj[4] = {1, 3};
```

9.5 Breadth-first search (BFS)

The slide deck states that BFS traverses breadth-wise and uses a queue to move to the next node.

```
void bfs(const vector<vector<int>>& adj, int start) {
    vector<bool> visited(adj.size(), false);
    queue<int> q;

    visited[start] = true;
    q.push(start);

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        cout << u << ' ';

        for (int v : adj[u]) {
            if (!visited[v]) {
                visited[v] = true;
                q.push(v);
            }
        }
    }
}
```

For the slide example, one shown BFS order is *A, B, C, D, E*.

BFS spreads outward in rings: every vertex at distance 1 is visited before any vertex at distance 2. The queue is what enforces that, because it hands back discovered vertices in the order they were found (Figure 9.3).

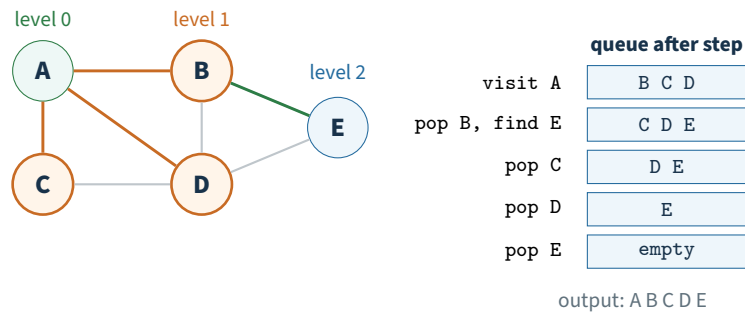


Figure 9.3. BFS from A, with the queue shown after each step. Vertices are shaded as they are discovered and outlined once processed. Because the queue is FIFO, the whole of level 1 is drained before E — the only level-2 vertex — is reached.

9.6 Depth-first search (DFS)

The slides state that DFS traverses depth-wise and uses a stack to reach the next nodes. Recursion is another way to obtain stack behavior.

```
void dfs(const vector<vector<int>>& adj,
        int u, vector<bool>& visited) {
    visited[u] = true;
    cout << u << ' ';

    for (int v : adj[u]) {
        if (!visited[v]) dfs(adj, v, visited);
    }
}
```

One DFS order shown by the deck is *A, B, E, D, C*.

DFS commits to one branch and follows it as far as it goes before considering any alternative. The stack holds the trail of vertices it may still need to back up through (Figure 9.4). Comparing it against Figure 9.3 on the same graph isolates the one thing that differs: a queue returns the *oldest* discovered vertex, a stack returns the *newest*.

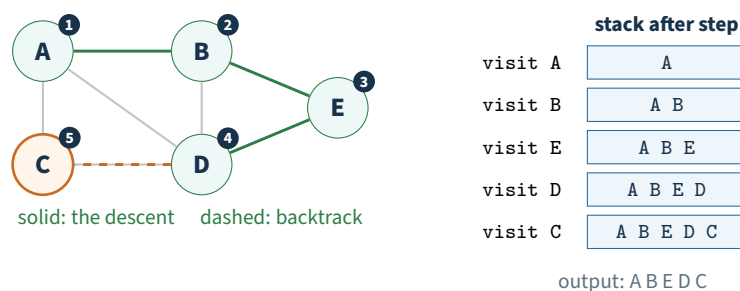


Figure 9.4. DFS from A, with the stack shown after each step. The walk plunges $A \rightarrow B \rightarrow E \rightarrow D$ before backtracking to pick up C. Swapping the queue in Figure 9.3 for a stack is the entire difference between the two algorithms.

Clarification beyond the slide wording

BFS and DFS traversal orders are not unique unless the neighbor-visiting order is fixed. The sequences in the slides are valid for a particular ordering of adjacent vertices; different adjacency-list orderings can produce different, equally valid traversals.

9.7 Why a visited set matters

Graphs may contain cycles. Without a visited marker, a traversal could keep moving around a cycle indefinitely.

Figure 9.5 shows the loop. In a tree, moving away from the root can never bring you back to it, so recursion terminates on its own. A graph has no such guarantee: the cycle $A \rightarrow B \rightarrow D \rightarrow A$ returns the traversal to a vertex it has already processed, and without a record of that fact the program will circle forever.

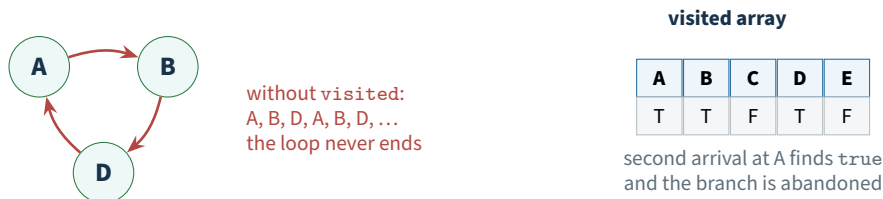


Figure 9.5. Why traversal needs a visited set. Following edges from A around the cycle leads back to A. The visited markers are the only thing that distinguishes “arrive at A for the first time” from “arrive at A again”, and therefore the only thing that terminates the walk.

Key idea

The queue or stack determines *which discovered vertex to process next*. The visited structure determines *whether a vertex should be processed again*.

9.8 Spanning trees

The slides define a spanning tree as a subset of a graph that covers all vertices using the minimum possible number of edges, has no cycles, and is connected.

For a connected graph with $|V|$ vertices, a spanning tree has $|V| - 1$ edges.

The lab then asks students to find a **minimum spanning tree** of an undirected graph using a greedy approach.

Clarification beyond the slide wording

The deck requests a greedy minimum-spanning-tree implementation but does not name a specific algorithm. Common textbook choices are Prim’s algorithm and Kruskal’s algorithm; naming them here is additional context, not material explicitly taught in the supplied slides.

Check your understanding

Why is an adjacency matrix often more wasteful for a sparse graph than an adjacency list? Which earlier ADT is directly reused by BFS, and which is reused by DFS?

Chapter recap

- A graph is represented as $G(V, E)$ with vertices and edges.
- Directed edges have orientation; undirected edges do not.
- Graphs can be stored with adjacency matrices or adjacency lists.
- BFS uses queue behavior; DFS uses stack behavior.
- A spanning tree is connected, acyclic, covers every vertex, and uses $|V| - 1$ edges.

PRACTICAL MILESTONE

Lab 9 of 11

Graph representation, traversal, and spanning trees

Source alignment — Official Lab Manual — Lab IX

Objective. Understand graph representation, implementation, and traversal.

Outcomes. Understand graph properties, representations, and BFS/DFS traversal.

Problem directions. Implement an adjacency list for m vertices and n edges; test whether a path of a given length exists; find a minimum spanning tree of an undirected graph using a greedy approach.

Build, traverse, and analyze one graph

Coverage. Chapter 9.

Source files. Lab07/Task02.cpp (adjacency list, BFS, DFS, path-length check), Task03.cpp (Prim's MST), and Task01.cpp (Dijkstra).

Goal. Build an adjacency list, traverse it with BFS and DFS, and find a minimum spanning tree of a weighted graph.

```
void addUndirectedEdge(vector<vector<int>>& adj, int u, int v) {  
    adj[u].push_back(v);  
    adj[v].push_back(u);  
}
```

Use the course graph or create a weighted undirected test graph. Implement adjacency-list storage, BFS, DFS, and the path-length check required by the manual.

Try it yourself

Reorder the adjacency list and rerun BFS/DFS. Explain why an exact visitation order may change. Then assign edge weights and apply the greedy MST method specified by your instructor.

Clarification beyond the slide wording

The course source asks for a greedy minimum-spanning-tree solution but does not choose Prim's or Kruskal's algorithm. State which algorithm you use and follow your instructor's specification if one is given.

Verification checklist

- Every valid edge appears in the adjacency structure with the intended directionality.
- BFS uses queue behavior; DFS uses recursion or an explicit stack.
- A visited structure prevents infinite traversal on cycles.
- The MST includes every vertex, has no cycle, and has $|V| - 1$ edges when the graph is connected.

Lab deliverable

Submit adjacency-list construction, BFS, DFS, the required path test, and a labelled MST result for a weighted undirected example.

Stage 5

Core Algorithms and Fast Access

Searching and Sorting

Chapter at a glance

Before you start: Know arrays, tree/heap concepts, and the Big-O notation used throughout earlier chapters.

By the end, you can: trace linear and binary search, distinguish internal and external sorting, explain bubble/insertion/selection/merge/quick/heap sort at the level of the slides, and compare their stated running-time behavior.

Searching retrieves an element from a data structure. Sorting rearranges data into a specified order. The Searching & Sorting deck places these topics together because sorted order can make some searches much faster.

10.1 Linear search

Linear search checks elements one by one until the target is found or the sequence ends.

```
int linearSearch(const vector<int>& a, int target) {
    for (int i = 0; i < static_cast<int>(a.size()); ++i) {
        if (a[i] == target) return i;
    }
    return -1;
}
```

The slides give linear search time complexity as $O(n)$ because the whole array may need to be traversed.

Worked example

Search for 30 in [12, 25, 18, 30, 22]. Compare with 12, 25, 18, then 30. The target is found at index 3 after four comparisons.

10.2 Binary search

Binary search requires the array to be sorted. It compares the target with the middle element, then discards the half that cannot contain the target.

```
int binarySearch(const vector<int>& a, int target) {
    int left = 0;
    int right = static_cast<int>(a.size()) - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (a[mid] == target) return mid;
        if (target < a[mid]) right = mid - 1;
        else left = mid + 1;
    }
    return -1;
}
```

The slides give binary search time complexity as $O(\log n)$.

Key idea

Binary search gains speed by using sorted order. Without sorted order, the “discard half” decision is not valid.

Figure 10.1 traces the collapse of the search window. Each comparison removes half of what remains, so a 16-element array needs at most 4 comparisons and a million-element array at most 20. That is the whole difference between $O(n)$ and $O(\log n)$.

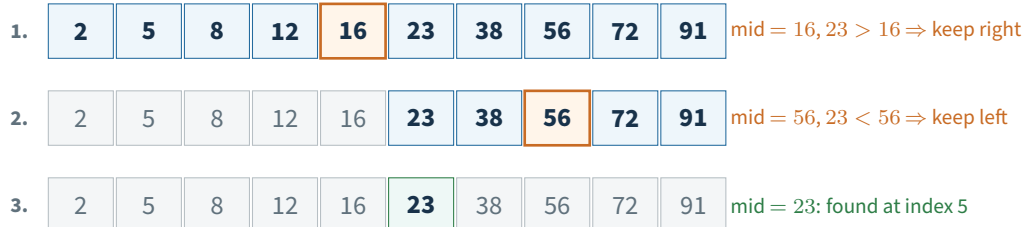


Figure 10.1. Binary search for 23 in a sorted 10-element array. Greyed cells have been eliminated without ever being examined — three comparisons rule out nine of the ten positions.

10.3 Sorting categories

The slide deck distinguishes two broad contexts.

Internal sorting All data to be sorted is placed in memory.

External sorting The full dataset cannot be placed in memory at once; the method must work with data that is partly external to memory.

The rest of the deck focuses on named sorting algorithms rather than external-sorting implementation details.

10.4 Bubble sort

Bubble sort is comparison-based. Adjacent elements are compared and swapped when they are out of order.

```
void bubbleSort(vector<int>& a) {
    int n = static_cast<int>(a.size());
    for (int pass = 0; pass < n - 1; ++pass) {
        for (int i = 0; i < n - 1 - pass; ++i) {
            if (a[i] > a[i + 1]) {
                swap(a[i], a[i + 1]);
            }
        }
    }
}
```

The slide states $O(n^2)$ time and notes that bubble sort is not suitable for large data sizes.

The name describes the motion: on each pass the largest remaining value is carried to the right-hand end, the way a bubble rises (Figure 10.2). After pass k the last k positions are finished, which is why the inner loop shrinks each time.

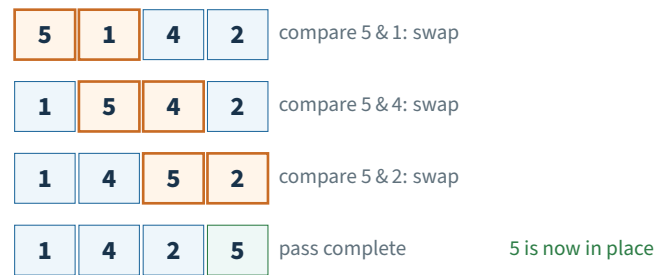


Figure 10.2. One full pass of bubble sort over $[5, 1, 4, 2]$. Each adjacent pair is compared and swapped when out of order; by the end of the pass the maximum, 5, has been pushed into its final position. Three more passes finish the array.

10.5 Insertion sort

Insertion sort maintains a sorted subarray and inserts the next element into its proper location in that sorted part.

```
void insertionSort(vector<int>& a) {
    for (int i = 1; i < static_cast<int>(a.size()); ++i) {
        int key = a[i];
        int j = i - 1;

        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            --j;
        }
        a[j + 1] = key;
    }
}
```

The slides state worst-case $O(n^2)$ and give $O(n)$ comparisons in the best case versus $O(n^2)$ in the worst case.

Figure 10.3 shows why the best case is so much better than the worst. The inner `while` loop stops as soon as it meets a smaller value, so on already-sorted input it never executes at all and the algorithm makes a single sweep. This is the same reason insertion sort is the method of choice for small or nearly-sorted arrays.

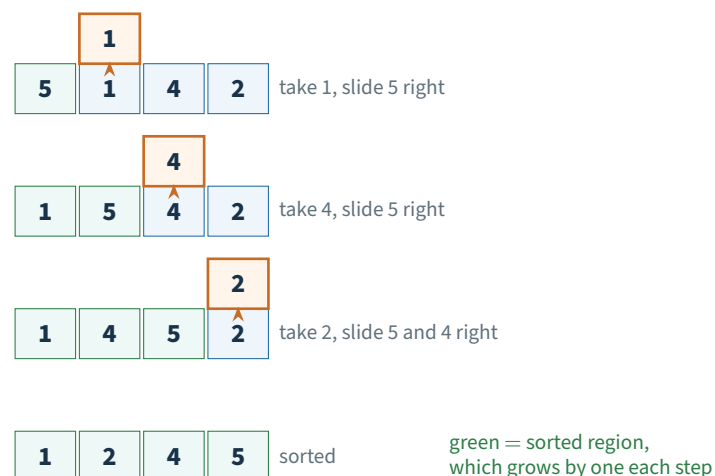


Figure 10.3. Insertion sort on $[5, 1, 4, 2]$. The green region on the left is always sorted; each step lifts the next value out and slides it back until it lands in order. The lifted value is shown above the array while the gap it left is being shifted.

10.6 Selection sort

Selection sort divides the list into a sorted part on the left and an unsorted part on the right. At each step, it finds the smallest element in the unsorted region and swaps it into the next sorted position.

```
void selectionSort(vector<int>& a) {
    int n = static_cast<int>(a.size());
    for (int i = 0; i < n - 1; ++i) {
        int minPos = i;
        for (int j = i + 1; j < n; ++j) {
            if (a[j] < a[minPos]) minPos = j;
        }
        swap(a[i], a[minPos]);
    }
}
```

The slide gives average and worst-case complexity $O(n^2)$ and says the method is not suitable for large datasets.

Insertion and selection sort both grow a sorted region on the left, but they pay for it differently (Figure 10.4). Selection sort scans the whole unsorted region to find the minimum, then performs exactly one swap; insertion sort takes whatever value comes next and pays with shifts. Selection sort therefore always makes $n - 1$ swaps regardless of the input, which matters when a swap is expensive.

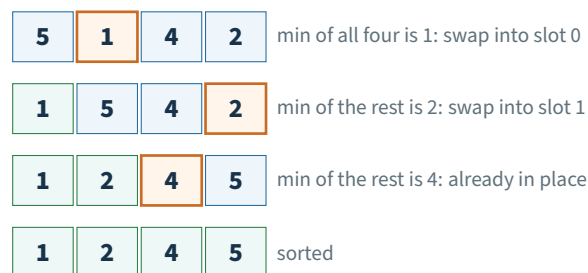


Figure 10.4. Selection sort on $[5, 1, 4, 2]$. Each pass scans the unsorted region for its smallest value and swaps it into the boundary position. Unlike insertion sort, the number of comparisons is the same whether the input is sorted or reversed.

10.7 Merge sort

Merge sort follows divide and conquer. The slide description repeatedly divides the array into two halves until subarrays contain one element, then merges sorted subarrays to build the complete sorted array.

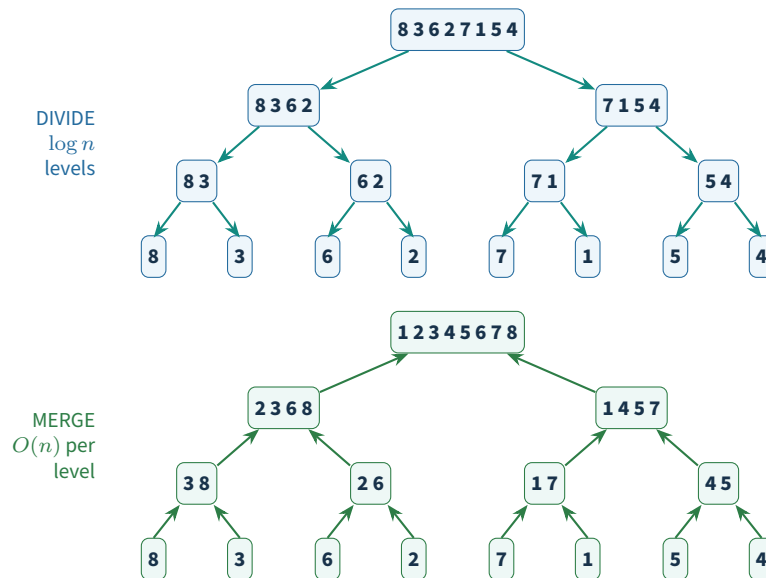


Figure 10.5. Merge sort on [8, 3, 6, 2, 7, 1, 5, 4]. The array is halved until every piece has one element — which is trivially sorted — and the pieces are then merged back in order. The tree has $\log n$ levels and each level does $O(n)$ work merging, which is where $O(n \log n)$ comes from.

Clarification beyond the slide wording

The supplied merge-sort slide explains the divide-and-conquer process but does not state a time complexity. Standard analysis gives $O(n \log n)$ time; that value is additional context rather than text copied from the slide.

10.8 Quick sort

Quick sort also follows divide and conquer. The deck describes partitioning the array around a **pivot**: values smaller than the pivot go to one side, values greater than the pivot to the other, then the resulting subarrays are sorted recursively.

```
int partition(vector<int>& a, int low, int high) {
    int pivot = a[high];
    int i = low - 1;

    for (int j = low; j < high; ++j) {
        if (a[j] <= pivot) {
            ++i;
            swap(a[i], a[j]);
        }
    }
    swap(a[i + 1], a[high]);
    return i + 1;
}

void quickSort(vector<int>& a, int low, int high) {
    if (low >= high) return;
    int p = partition(a, low, high);
    quickSort(a, low, p - 1);
    quickSort(a, p + 1, high);
}
```

Figure 10.6 traces one partition step. The pivot ends up at the index that splits the array into “everything smaller” and “everything larger”, so the pivot is placed correctly and never moves again. Quick sort then recurses on the two sides.

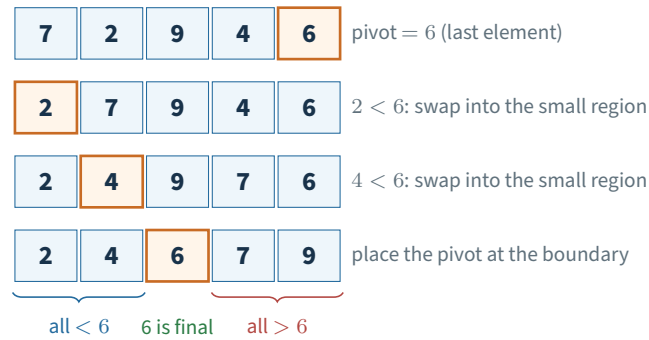


Figure 10.6. Partitioning [7, 2, 9, 4, 6] around the pivot 6. The pointer *i* marks the end of the “smaller than pivot” region; each value at *j* that is smaller is swapped up into it. The final swap drops the pivot into the boundary, at which point index 2 holds its permanently correct value.

Clarification beyond the slide wording

The quick-sort slide says its “average and worst-case complexities are $O(n^2)$, respectively,” and gives best-case $O(n \log n)$. Standard algorithm analysis is: best $O(n \log n)$, average $O(n \log n)$, and worst $O(n^2)$. The discrepancy is surfaced explicitly here rather than silently changing the source.

10.9 Heap sort

The heap-sort slide connects sorting with the heap from Trees II. It describes building a min-heap or max-heap from the array, then repeatedly eliminating elements from the heap portion and inserting them into the sorted portion.

The slide states that heap sort is in-place and has time complexity $O(n \log n)$.

```
void heapify(vector<int>& a, int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && a[left] > a[largest]) largest = left;
    if (right < n && a[right] > a[largest]) largest = right;

    if (largest != i) {
        swap(a[i], a[largest]);
        heapify(a, n, largest);
    }
}

void heapSort(vector<int>& a) {
    int n = static_cast<int>(a.size());
    for (int i = n / 2 - 1; i >= 0; --i)
        heapify(a, n, i);

    for (int end = n - 1; end > 0; --end) {
        swap(a[0], a[end]);
        heapify(a, end, 0);
    }
}
```

10.10 Why the growth rate is what matters

Complexity classes are easy to treat as labels attached to algorithms. Figure 10.7 shows what they actually claim. For a handful of items the difference is invisible — at $n = 10$ every one of these curves is small. The classes separate as n grows, and they separate violently.

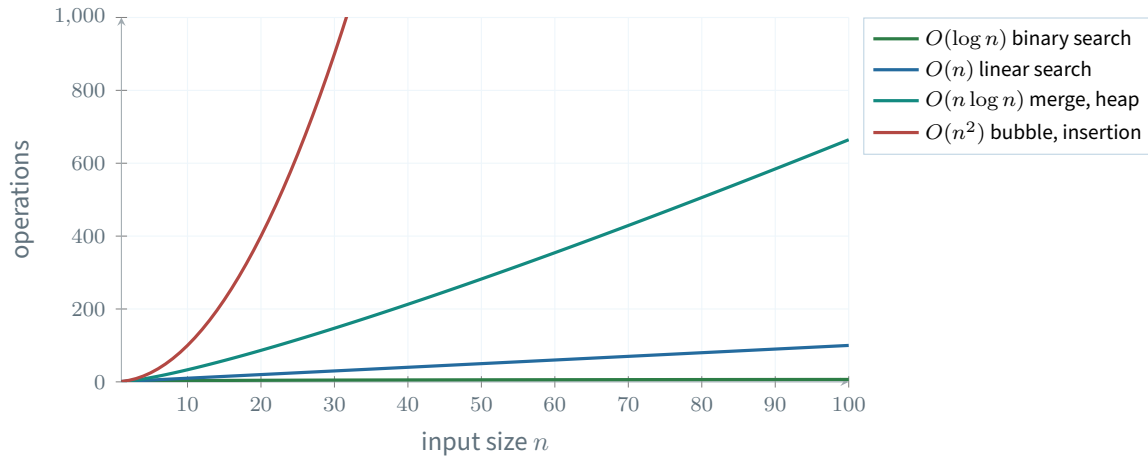


Figure 10.7. Operation counts for the complexity classes in this chapter. At $n = 100$ a quadratic algorithm already does roughly 15 times the work of an $n \log n$ one; at $n = 1000$ it does about 100 times as much. This is why bubble sort and merge sort are not interchangeable even though both sort correctly.

Key idea

Constant factors and hardware speed shift these curves up or down; they do not change their shape. A faster computer running an $O(n^2)$ sort still loses to a slower computer running an $O(n \log n)$ sort once n is large enough — and “large enough” arrives sooner than most people expect.

10.11 Algorithm comparison

Algorithm	Complexity stated in slides	Main slide idea
Linear search	$O(n)$	Check items one by one.
Binary search	$O(\log n)$	Sorted input; repeatedly choose one half.
Bubble sort	$O(n^2)$	Swap adjacent out-of-order elements.
Insertion sort	Best comparisons $O(n)$; worst $O(n^2)$	Insert into a growing sorted subarray.
Selection sort	Average/worst $O(n^2)$	Select smallest remaining item.
Merge sort	Not stated	Divide, sort halves, merge.
Quick sort	Slide wording has a complexity inconsistency	Partition around a pivot and recurse.
Heap sort	$O(n \log n)$	Use heap ordering to extract elements.

10.11.1 Standard analysis for reference

The table above records what the slides state. The one below gives the conventional textbook analysis of the same algorithms, including two properties the slides do not discuss: extra space, and **stability** — whether equal elements keep their original relative order, which matters when sorting records by a secondary key.

Algorithm	Best	Average	Worst	Space	Stable
Bubble sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	yes
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	yes
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	no
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	yes
Quick sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	no
Heap sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	no

Clarification beyond the slide wording

This table is standard algorithm analysis, not slide content. It is included because the slide deck states complexities inconsistently for quick sort and omits merge sort entirely. Where the two tables disagree, the discrepancy is the point: check which convention your assessment expects.

Intuition

Bubble and insertion sort reach $O(n)$ in the best case only because they can detect that the data is already ordered and stop early. Selection sort cannot: it must scan the entire unsorted region to be sure it has found the minimum, so its best case is no better than its worst.

Check your understanding

Why does binary search require sorted data? Explain in one sentence how insertion sort and selection sort differ in the way they grow the sorted portion.

Chapter recap

- Linear search scans; binary search repeatedly halves a sorted search range.
- Internal sorting fits the dataset in memory; external sorting addresses data too large to fit at once.
- Bubble, insertion, and selection sort are comparison-based methods with quadratic worst-case behavior in the slides.
- Merge sort and quick sort use divide and conquer.
- Heap sort reuses the heap structure and is stated as $O(n \log n)$.
- Complexity claims must be read with their assumptions; the quick-sort slide discrepancy is explicitly noted.

PRACTICAL MILESTONE

Lab 10 of 11

Searching and sorting experiments

Source alignment — Official Lab Manual — Lab X

Objective. Search a list and perform sorting using different techniques.

Outcomes. Understand searching techniques and the working of multiple sorting techniques.

Manual problems.

1. Sort an array in descending order using quick sort, always selecting the median element of the current array/subarray as pivot.
2. Implement binary search and test whether a given element is present.
3. Modify merge logic to merge three sorted arrays into one array.
4. Sort an array using heap sort.

Trace, implement, and compare algorithms

Coverage. Chapter 10.

Source files. Lab08/Task05 .cpp (binary search), Task06 .cpp (descending quick sort, median pivot), Task07 .cpp (three-way merge), and Task08 .cpp (heap sort). Task01–Task04 are bubble, insertion, quick, and selection sort.

Goal. Implement binary search and several sorting methods, then compare them on one common dataset.

Use a common dataset where appropriate, for example:

```
vector<int> data = {29, 10, 14, 37, 13};
```

Try it yourself

Trace at least two sorting methods by hand. Then implement the four manual problems above. Count comparisons or swaps for one sorting pair so the final-project style of algorithm comparison becomes familiar.

Clarification beyond the slide wording

The source's quick-sort theory contains a complexity inconsistency, documented in Chapter 10. Also, “median element as pivot” can mean the element at the middle index rather than the statistical median value; follow the wording used by your instructor or clarify the interpretation before coding.

Verification checklist

- Binary search is run only on sorted input.
- Descending quick sort produces a correctly ordered final array.
- The three-way merge preserves every input value exactly once.
- Heap sort output is sorted and the heap property is maintained during extraction.

- Test cases include duplicates and a missing search target.

Lab deliverable

Submit implementations for the assigned manual problems, sample output, and at least one complete trace showing intermediate states.

Hashing

Chapter at a glance

Before you start: Know direct array indexing, searching, and why reducing search time is valuable.
By the end, you can: explain a hash function, trace a modular hash, identify collisions, and implement the three collision-resolution techniques named in the slides: linear probing, quadratic hashing, and double hashing.

Hashing is used for fast access to elements. The slide deck describes the core idea as converting a key into a smaller value I with a **hash function**, then using I as an index where the key is stored or searched.

11.1 Hash function and table index

A simple modular hash function has the form

$$h(k) = k \bmod m,$$

where m is the number of table slots.

The slide's hashing example uses $H(x) = x \bmod 5$ for keys 10, 12, and 14.

Worked example

$$10 \bmod 5 = 0, \quad 12 \bmod 5 = 2, \quad 14 \bmod 5 = 4.$$

So the keys occupy indices 0, 2, and 4.

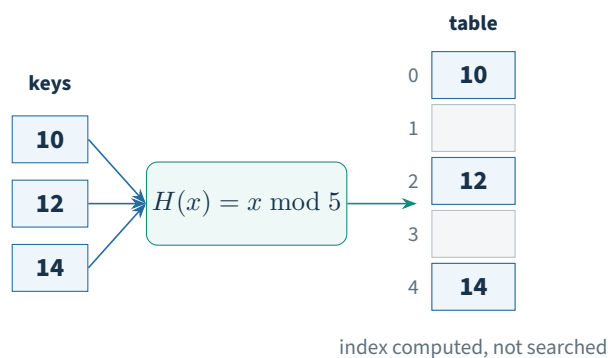


Figure 11.1. The hash function maps an unbounded key space onto a small fixed range of table indices. Because the arrow is computed rather than searched for, a lookup goes straight to the slot instead of scanning.

The slides state that a simple modular hash calculation can be $O(1)$ and therefore supports very efficient storage and access in the intended hashing model.

Clarification beyond the slide wording

The slide's $O(1)$ statement describes the intended constant-time hash calculation and typical fast table access. Actual access time depends on collision handling and table load; heavy collisions can require multiple probes.

11.2 Collision

A **collision** occurs when two different keys produce the same hash value and therefore target the same table slot.

The slide example asks what happens when key 15 is inserted:

$$15 \bmod 5 = 0,$$

which conflicts with key 10 already at index 0.

Figure 11.2 shows the conflict. The hash function compresses a large key space into m slots, so once more than m distinct keys are in play collisions are not merely likely — they are unavoidable. Every practical hash table therefore needs a rule for what to do when the target slot is taken.



Figure 11.2. A collision. Keys 10 and 15 both hash to index 0, because $10 \bmod 5$ and $15 \bmod 5$ are both zero. The hash function cannot be blamed and cannot be fixed: any function mapping many keys onto five slots must send some of them to the same place.

The deck names three collision-handling techniques:

- linear probing;
- double hashing;
- quadratic hashing.

11.3 Linear probing

Linear probing checks the next slot, then the next, wrapping around the table if required.

A probe sequence can be written as

$$(h(k) + i) \bmod m, \quad i = 0, 1, 2, \dots$$

Worked example

With $m = 5$, key 15 hashes to 0. If index 0 is occupied, linear probing tries 1, then 2, then 3, then 4 until an empty slot is found.

Figure 11.3 traces a sequence of insertions and shows the characteristic weakness of linear probing: **clustering**. Because every displaced key lands immediately next to the block it collided with, occupied slots grow into runs, and each new collision must walk the whole run before finding space.

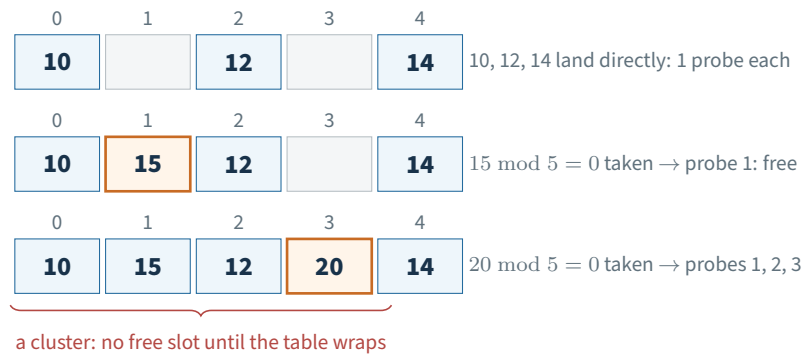


Figure 11.3. Inserting 10, 12, 14, 15, 20 into a five-slot table with $H(x) = x \bmod 5$ and linear probing. Keys 15 and 20 both hash to slot 0 and must walk forward, forming a cluster. By the last insertion a single probe has become four.

```
int insertLinear(vector<int>& table, int key, int EMPTY) {
    int m = static_cast<int>(table.size());
    int start = key % m;

    for (int i = 0; i < m; ++i) {
        int idx = (start + i) % m;
        if (table[idx] == EMPTY) {
            table[idx] = key;
            return idx;
        }
    }
    return -1; // no free slot
}
```

11.4 Quadratic hashing / quadratic probing

Quadratic probing changes the step size quadratically, for example

$$(h(k) + i^2) \bmod m.$$

```
int insertQuadratic(vector<int>& table, int key, int EMPTY) {
    int m = static_cast<int>(table.size());
    int h = key % m;

    for (int i = 0; i < m; ++i) {
        int idx = (h + i * i) % m;
        if (table[idx] == EMPTY) {
            table[idx] = key;
            return idx;
        }
    }
    return -1;
}
```

Clarification beyond the slide wording

The slide calls the technique “quadratic hashing.” In common data-structures terminology, the collision-resolution method is usually called *quadratic probing*. The notes keep both labels visible.

11.5 Double hashing

Double hashing uses one hash function for the initial position and another to choose the probe step.

One illustrative form is

$$\text{index}_i = (h_1(k) + i h_2(k)) \bmod m.$$

```
int insertDoubleHash(vector<int>& table, int key, int EMPTY) {
    int m = static_cast<int>(table.size());
    int h1 = key % m;
    int h2 = 1 + (key % (m - 1));

    for (int i = 0; i < m; ++i) {
        int idx = (h1 + i * h2) % m;
        if (table[idx] == EMPTY) {
            table[idx] = key;
            return idx;
        }
    }
    return -1;
}
```

Clarification beyond the slide wording

The hashing deck names double hashing but does not specify a second hash function. The formula above is an implementation example for the lab, not a formula taken from the slide.

11.6 Searching under open addressing

The probe rule used for insertion must also be used for searching. Otherwise the program may stop before reaching the slot where a collided key was placed.

Key idea

A hash table does not eliminate the need for searching logic; it changes the search space from “scan the whole collection” to “calculate an index, then follow the collision-resolution probe sequence if necessary.”

11.7 Separate chaining as an alternative

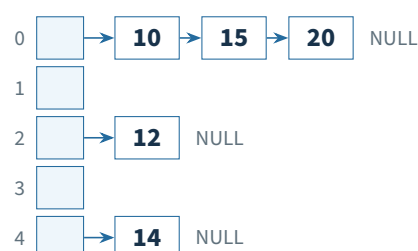
All three techniques above are forms of **open addressing**: every key lives in the table itself, so the table can fill up and clustering degrades every probe. The main alternative stores a linked list at each slot instead, letting colliding keys share a slot rather than compete for others (Figure 11.4).

Open addressing



table full at 5 keys;
probes lengthen as it fills

Separate chaining



never full; lookup walks one short list

Figure 11.4. Separate chaining. Colliding keys are appended to a list hanging off their slot, so an insertion never disturbs another slot and the table cannot overflow. The cost is one pointer per entry and a short list walk on lookup.

Clarification beyond the slide wording

Separate chaining is not named in the supplied hashing slides, which cover linear probing, quadratic hashing, and double hashing. It is included here because it is the standard alternative and because it explains the deletion problem described in the next section: a chained table has no tombstone issue, since removing a node from a list leaves no gap in a probe sequence.

11.8 Deletion caution

With open addressing, simply marking a deleted slot as never-used can incorrectly terminate a later search whose probe sequence must pass through that slot.

Clarification beyond the slide wording

Deletion behavior and tombstone markers are not covered in the supplied hashing slides. This caution is included only to explain why a complete open-addressing implementation needs more than insertion logic.

Check your understanding

Using $H(x) = x \bmod 5$, where do keys 10, 12, 14, and 15 initially hash? Which pair collides? How does linear probing react?

Chapter recap

- A hash function converts a key into a table index.
- The slide example uses modular hashing and highlights $O(1)$ hash calculation / fast-access intent.
- A collision occurs when multiple keys map to the same slot.
- Linear probing checks consecutive slots.
- Quadratic probing changes the probe distance quadratically.
- Double hashing uses a second hash function to determine the probe step.

PRACTICAL MILESTONE

Lab 11 of 11

Hashing and collision resolution

Source alignment — Official Lab Manual — Lab XI

Objective. Learn hashing techniques for efficient storage and access.

Outcome. Store data so it can be accessed efficiently.

Problem directions. Practice hashing and resolve collisions using linear probing, double hashing, and quadratic hashing/probing.

Resolve the same collision three ways

Coverage. Chapter 11.

Source files. Lab09/Task01.cpp — linear probing, quadratic probing, and double hashing on one table.

Goal. Resolve the same collisions three ways — linear probing, quadratic probing, and double hashing — and record every probe.

Use the source example $H(x) = x \bmod 5$ with keys 10, 12, and 14, then insert 15 so the home slot 0 collides with 10.

```
vector<int> keys = {10, 12, 14, 15};  
const int TABLE_SIZE = 5;
```

Try it yourself

For each collision strategy, record every probe index rather than only the final slot. Then add more keys and compare how probe sequences grow as the table becomes crowded.

Verification checklist

- Every inserted key remains discoverable using the same probe rule used for insertion.
- Probe sequences stay inside the table using modular arithmetic.
- The double-hash step does not collapse to zero and is suitable for the chosen table size.
- The term “quadratic hashing” in the source is connected to the usual term “quadratic probing.”

Lab deliverable

Submit all three collision-resolution methods and a comparison table listing the probe sequence and final slot for each key.

Stage 6

Course Project and Synthesis

Final Project: Algorithm Comparison

Chapter at a glance

Before you start: Know how to trace algorithms, state time/space complexity, and implement representative course algorithms in C++.

By the end, you can: choose a valid pair of related algorithms, design a fair comparison, instrument operation counts and runtime, build the required interactive app, and structure the written report around evidence rather than claims.

The final-project guideline turns the course into a small comparative experiment. The required deliverable is not only a written report: the group must also build an interactive web or mobile application that demonstrates two algorithms from the *same category* side by side.

Final-project requirement

Choose two related algorithms or operations, explain how both work, and compare their time complexity, space complexity, number of steps or operations, use cases, and practicality. The application must accept sample input, execute both choices, show the results, report steps and measurable time, provide visual output, and present a side-by-side comparison.

12.1 What counts as a valid comparison?

The project guideline explicitly requires algorithms from the same category. It gives the following examples.

Category	Example pair	Course connection
Sorting	Bubble Sort vs Merge Sort	Chapter 10
Searching	Linear Search vs Binary Search	Chapter 10
Data-structure operations	Array insertion vs linked-list insertion	Chapters 2 and 3
Graph traversal	BFS vs DFS	Chapter 9
Recursion vs iteration	Factorial or Fibonacci implemented both ways	General algorithm-design comparison

The guideline also permits other pairs, provided the two choices are genuinely related. A comparison such as Bubble Sort vs Binary Search is invalid because the algorithms solve different problems.

Recommended workflow (added guidance)

A strong pair has one shared task and one clear independent variable: the algorithm. Keep the input, hardware, programming language, output requirement, and measurement method the same. That makes the comparison easier to explain and more defensible.

12.2 Required comparison dimensions

For each algorithm, the report is expected to cover how it works, a step-by-step example, best/average/worst time complexity, space complexity, and pros/cons. The comparison itself should also discuss step counts, use cases, and practicality.

Dimension	What to collect
Working principle	A concise description of the rule each algorithm follows.
Correctness evidence	Identical expected output for the same valid input, or an explanation of why outputs differ.
Time complexity	Best, average, and worst cases where meaningful. State assumptions.
Space complexity	Auxiliary memory and any important representation cost.
Operation count	Comparisons, swaps, probes, node visits, queue/stack operations, or another pair-specific count.
Measured runtime	Repeated timing under the same conditions; treat small-input timings cautiously.
Practicality	Input constraints, setup cost, ease of implementation, and when one method is a better fit.
Use cases	One or two scenarios where the trade-off matters.

12.3 Counting operations in C++

An operation counter makes the comparison visible even when runtime is too small or noisy to interpret. The exact counter should match the algorithm.

```
struct Metrics {
    long long comparisons = 0;
    long long swaps = 0;
    long long visits = 0;
};

void bubbleSortMeasured(vector<int>& a, Metrics& m) {
    int n = static_cast<int>(a.size());
    for (int pass = 0; pass < n - 1; ++pass) {
        for (int i = 0; i < n - 1 - pass; ++i) {
            ++m.comparisons;
            if (a[i] > a[i + 1]) {
                swap(a[i], a[i + 1]);
                ++m.swaps;
            }
        }
    }
}
```

Key idea

Do not force every algorithm into the same notion of a “step.” Define the metric before the experiment. For sorting, comparisons and swaps are natural. For searching, comparisons are natural. For graphs, visited vertices/edges and queue/stack operations may be more meaningful.

12.4 Measuring runtime fairly

The project asks for time taken *if measurable*. C++ can use `std::chrono`; however, one very short execution can be dominated by timer resolution, process scheduling, caching, or startup effects.

```
#include <chrono>
using namespace std::chrono;

auto start = high_resolution_clock::now();
algorithm(input);
auto stop = high_resolution_clock::now();

auto elapsed = duration_cast<microseconds>(stop - start).count();
cout << elapsed << " microseconds\n";
```

Recommended workflow (added guidance)

Run each algorithm multiple times on copied input and report a median or average. For sorting, never let Algorithm 2 receive the already-sorted output produced by Algorithm 1 unless “sorted input” is intentionally the scenario being tested. Preserve an untouched master input and copy it before each run.

12.5 Application requirements

The final-project guideline requires a web or mobile application before report submission. The interface should help even a non-computer-science user interpret the comparison.

Final-project requirement

The app must allow sample-data input, execute both algorithms, display results, show step counts, show time taken when measurable, include visual outputs, and provide a side-by-side comparison. The guideline specifically encourages interactive elements such as dropdowns and buttons.

A practical screen flow is:

- Step 1:** Choose the algorithm pair or open the fixed project pair.
- Step 2:** Enter or generate the input dataset.
- Step 3:** Validate the input and show any precondition, such as “binary search requires sorted data.”
- Step 4:** Run Algorithm 1 and Algorithm 2 on equivalent copies of the input.
- Step 5:** Display result, operation count, elapsed time, and a compact visual trace.
- Step 6:** Show a comparison table and one plain-language interpretation.

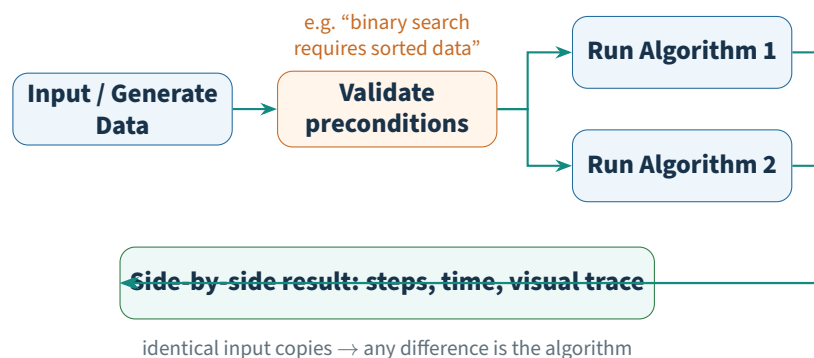


Figure 12.1. Screen flow for the comparison application. The two algorithms must receive equivalent copies of the same input, or the measured difference reflects the data rather than the algorithms.

12.6 Report structure required by the guideline

The written report is organized into six main parts.

A. Introduction Identify the algorithms and explain why the comparison is useful or interesting.

B. Algorithm Explanation

Explain each method, give a step-by-step example, state best/average/worst time and space complexity, and discuss pros/cons.

C. Comparison Table

Compare working principle, time, space, number of steps, and best use cases side by side.

D. Use Case Comparison

Give one or two real-world scenarios and explain why one algorithm is more suitable in each.

E. Visual Aid (optional)

Add flowcharts, diagrams, operation-count graphs, or screenshots from the application.

F. Conclusion

Summarize where each algorithm is more efficient or practical and state the group's takeaways.

12.7 Team and repository requirements

Final-project requirement

Teams may contain a maximum of four students. Each group submits one report and one application. The team leader should upload the code to a GitHub repository, give other team members access, reference members there, and include the repository in the report so the code can be accessed.

Recommended workflow (added guidance)

Use a simple repository structure such as `/app`, `/algorithm-core`, `/tests`, `/docs`, and a clear `README`. Keep algorithm logic separate from UI code so the same functions can be tested independently and then called by the web/mobile interface.

12.8 Project pair ideas from this course

The following pairs stay within the same problem category and have useful trade-offs to visualize.

Pair	Useful evidence to visualize
Linear vs binary search	Comparisons as n grows; binary-search precondition; target position.
Bubble vs insertion sort	Comparisons and swaps for random, sorted, and nearly sorted input.
Bubble vs merge sort	Quadratic growth vs divide-and-conquer behavior; extra merge storage.
BFS vs DFS	Visit order, frontier data structure, discovered path behavior.
Array vs linked-list insertion	Shifts vs pointer updates, plus the cost of locating the insertion position.
Linear vs quadratic probing	Probe sequences and clustering after the same collisions.
Min-heap vs max-heap operations	Heap restoration path and which extreme is available at the root.

12.9 Submission checklist

Verification checklist

Before submission, verify that the package contains:

- project title and complete group details;
- introduction to the selected pair;
- detailed explanation and code for both algorithms;
- step-by-step examples and complexity analysis;
- a comprehensive side-by-side comparison table;
- real-world use-case analysis;
- a conclusion supported by the collected evidence;
- the functioning web/mobile application;
- the GitHub repository reference in the report;
- optional diagrams, graphs, and application screenshots where they improve interpretation.

Check your understanding

If one algorithm appears faster for a single input of size 20, is that enough to conclude it has better asymptotic time complexity? What additional evidence would make the comparison more convincing?

Chapter recap

- The project compares two related algorithms or operations from the same category.
- The comparison must include time, space, operation counts, practicality, and use cases.
- A web or mobile application is a required deliverable and must execute both choices interactively.
- Fair experiments keep inputs and measurement conditions comparable.
- The report structure and GitHub workflow are explicitly specified in the project guideline.

Course finish line

Recommended workflow (added guidance)

By the end of the course, aim to be able to do five things with any structure or algorithm you have studied: draw its state, trace one operation by hand, implement the core operation, state the relevant time/space cost with assumptions, and design a test that exposes an edge case. The final project combines all five skills in one comparison.

Quick Reference

Chapter at a glance

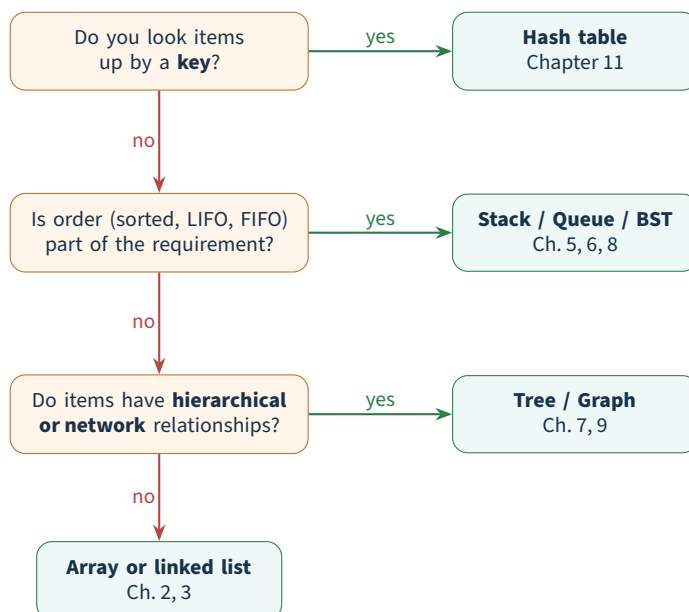
Before you start: Everything in this appendix has appeared earlier with full explanation; nothing here is new material.

By the end, you can: revise before an assessment, choose a structure for the final project, and check a complexity claim quickly.

This appendix collects the numbers and decisions scattered across the previous chapters into one place. Use it for revision, not for first learning: each entry points back to the chapter that explains it.

A.1 Choosing a data structure

Most structure choices in this course reduce to a short series of questions about the access pattern (Figure A.1). The pattern matters more than the data: the same student records belong in an array, a BST, or a hash table depending entirely on how they will be looked up.



Then choose between them on one further question: is the size fixed and is direct index access needed (array), or will there be many insertions and deletions (linked list)?

Figure A.1. A decision path for the structures covered in this course. Read it as a set of questions about how the data will be *used*, not about what the data is.

A.2 Operation costs by structure

Structure	Access	Search	Insert/Delete	Note
Array	$O(1)$	$O(n)$	$O(n)$	Shifting dominates insertion
Sorted array	$O(1)$	$O(\log n)$	$O(n)$	Binary search applies
Singly linked list	$O(n)$	$O(n)$	$O(1)$	Cost is $O(1)$ <i>once positioned</i>
Doubly linked list	$O(n)$	$O(n)$	$O(1)$	Deletion needs no predecessor search
Stack	$O(1)$ top	—	$O(1)$	Access restricted to the top
Queue	$O(1)$ ends	—	$O(1)$	Circular form reuses slots
BST (balanced)	—	$O(\log n)$	$O(\log n)$	Degrades to $O(n)$ if unbalanced
Heap	$O(1)$ root	$O(n)$	$O(\log n)$	Root is min or max only
Hash table	—	$O(1)$ avg	$O(1)$ avg	Worst case $O(n)$ under heavy collision

Clarification beyond the slide wording

These are standard averages for the implementations built in this course, not figures quoted from the slide decks. Where a chapter states a different value, the chapter's own clarification box explains the discrepancy.

A.3 Sorting algorithms at a glance

Algorithm	Average	Worst	Stable	Choose it when
Bubble	$O(n^2)$	$O(n^2)$	yes	Teaching only; rarely practical
Insertion	$O(n^2)$	$O(n^2)$	yes	Small or nearly-sorted input
Selection	$O(n^2)$	$O(n^2)$	no	Swaps are expensive; only $n - 1$ occur
Merge	$O(n \log n)$	$O(n \log n)$	yes	Guaranteed bound and stability needed
Quick	$O(n \log n)$	$O(n^2)$	no	Fast in practice; in-place
Heap	$O(n \log n)$	$O(n \log n)$	no	Guaranteed bound with $O(1)$ extra space

A.4 Traversal orders

Traversal	Rule	Typical use
Inorder	left, root, right	Sorted output from a BST
Preorder	root, left, right	Copying or serialising a tree
Postorder	left, right, root	Deleting or freeing a tree safely
BFS	queue, level by level	Shortest path in an unweighted graph
DFS	stack or recursion	Cycle detection, connectivity, backtracking

A.5 Pointer operations that go wrong most often

Mistake	What to do instead
Moving head before linking the new node	Set <code>n->next = head</code> first (Figure 3.2)
Deleting a node before re-linking its predecessor	Redirect <code>prev->next</code> , then delete (Figure 3.3)
Using <code>while (p != nullptr)</code> on a circular list	Use <code>do-while</code> against the head (Figure 4.2)
Popping or peeking without an empty check	Test <code>isEmpty()</code> before touching the top
Treating a full linear queue as genuinely full	Use modular arithmetic (Figure 6.3)
Omitting the visited set in graph traversal	Mark on discovery, not on processing (Figure 9.5)